

Formal Verification of FiniteKernels.jl

Simon Frost

2026-09-30

Contents

FiniteKernelsProofs	1
What is formalised	1
Correspondence with the Julia API	1
FiniteKernelsProofs.Finite.Kernel	2
Conventions	2
FiniteKernelsProofs.Finite.Laws	9
FiniteKernelsProofs.Theory.Correspondence	12
FiniteKernelsProofs.Theory.FinStoch	14
Column-major multi-axis arrays	17
Flat tables	21
The outputs-first kernel layout and the (parents..., child) CPT layout	22
Outputs-first kernel tables	23
The (parents..., child) CPT layout and cpt	24
Julia's permutedims contract	25
Normalisation	27
The stride-based product of two factors	27
Coordinates and projections	28
Strides	28
The odometer loop of _product_into!	30
Roadmap	32

FiniteKernelsProofs

Lean 4 / Mathlib formalisation accompanying FiniteKernels.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every library module is built by `lake build --wfail`; `Audit.lean` prints the axioms of the headline results (only `propext`, `Classical.choice`, `Quot.sound`). The Roadmap has no remaining proof holes.

What is formalised

FiniteKernels.jl implements finite stochastic kernels (FiniteKernel, tables in the outputs-first layout of ADR 0002), their sequential (`compose_kernel`) and parallel (`tensor_kernel`) composition and copy/discard/swap operations. MarkovCategories.jl exposes these as an instance of the GATlab theory `ThMarkovCategory` (SPEC §5–§6). The exact kernel equations are proved pointwise and are now bundled into a Mathlib MarkovCategory `FinStoch`, including the symmetric monoidal coherence laws. No external Stoch development is vendored and no correspondence with floating-point code is asserted.

- `Finite/Kernel.lean`: the finite model $\text{Kernel } X \ Y := X \rightarrow Y \rightarrow \mathbb{R}$, its operations, preservation of normalisation/nonnegativity, and the category, symmetric-monoidal and commutative-comonoid laws.
- `Finite/Laws.lean`: discard naturality iff normalised; copy naturality iff deterministic; “copy once is not two samples” for states.
- `Theory/Correspondence.lean`: the GATlab/Mathlib/finite-kernel dictionary, with examples checking the type of each Mathlib field.
- `Theory/FinStoch.lean`: concrete Mathlib category, monoidal, symmetric, comonoid and Markov instances; abstract copy naturality iff the kernel is deterministic. Empty objects are permitted without assuming nonexistent maps into them.
- `Roadmap.lean`: compatibility import and remaining representation questions; no unproved declarations.

Correspondence with the Julia API

Julia (<code>FiniteKernels.jl</code>)	Lean
<code>FiniteKernel table table[y..., x...],</code> <code>kernel_matrix(k)[y, x]</code>	<code>Kernel X Y, entry k x y = P(y x)</code>
<code>is_normalized(k)</code>	<code>Kernel.Normalised k</code>
<code>compose_kernel(k, l) (diagrammatic order)</code>	<code>Kernel.comp k l</code>
<code>tensor_kernel(k, l)</code>	<code>Kernel.tensor k l</code>
<code>identity_kernel, copy_kernel,</code> <code>discard_kernel, swap_kernel</code>	<code>idK X, copy X, discard X, swap X Y</code>
<code>deterministic(X, Y, f), point_mass(X, a),</code> <code>state(X, p)</code>	<code>ofFun f, pointMass a, State X</code>
<code>testset “category laws”</code>	<code>comp_assoc, idK_comp, comp_idK</code>
<code>testset “monoidal laws”</code>	<code>comp_tensor, tensor_idK, tensor_assoc,</code> <code>tensor_unit_left, tensor_unit_right, swap_swap,</code> <code>tensor_swap, hexagon</code>
<code>testsets “comonoid laws”, “coherence of copy and</code> <code>discard”</code>	<code>copy_assoc, copy_discard_left, copy_discard_right,</code> <code>copy_swap, copy_prod, discard_prod, copy_unit,</code> <code>discard_unit</code>
<code>“discard naturality iff normalised”</code>	<code>comp_discard_eq_discard_iff</code>
<code>“copy is natural for deterministic kernels only”</code>	<code>comp_copy_eq_iff_isDeterministic</code>
<code>“copy once is not two samples” (SPEC §6)</code>	<code>comp_copy_eq_tensor_iff_isPointMass</code>

SPEC §61’s Propositions 1–7 concern Bayesian networks and influence diagrams and are formalised in `BayesianNetworks.jl/proofs` and `InfluenceDiagrams.jl/proofs`; this project supplies the kernel-level laws those build on. The generic consequences of the abstract axioms (`discard_natural` as a theorem, `state_discard`, `deterministic_comp`, `deterministic_copy`) are stated once, for Mathlib’s classes, in `BayesianNetworksProofs.Markov.Basic`.

```
namespace FiniteKernelsProofs

/-- Smoke lemma so that the axiom audit always has a first line. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl

end FiniteKernelsProofs
```

FiniteKernelsProofs.Finite.Kernel

```
import Mathlib.Data.Real.Basic
import Mathlib.Algebra.BigOperators.Ring.Finset
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Fintype.Prod
import Mathlib.Logic.Equiv.Basic
import Mathlib.Algebra.Order.BigOperators.Group.Finset
import Mathlib.Tactic.Ring
```

A concrete finite model of the structure implemented by `FiniteKernels.jl` (`src/kernels.jl`, `src/composition.jl`, `src/copy_discard.jl`): finite stochastic kernels between finite types, with sequential composition, tensor product, copy, discard and swap.

The laws here are stated directly about kernels, pointwise, with the monoidal structure maps (associator, unitors, tensor μ) written as *function-reindexing kernels* of `Fun` e for the obvious equivalences. `Theory/FinStoch.lean` packages them into `Mathlib` category instances.

Conventions

- Kernel $X \rightarrow Y := X \rightarrow Y \rightarrow \mathbb{R}$; $k \ x \ y$ is the Julia entry $P(y \mid x)$, i.e. `table[y..., x...]` in the outputs-first layout of ADR 0002 (`kernel_matrix(k)[y, x]`).
- Normalised k is `is_normalized(k)`: every column $k \ x$ sums to one.
- Entries live in $\mathbb{R}$ rather than $\mathbb{R}_{\geq 0}$; nonnegativity is the separate predicate `Nonneg` and is preserved by every construction, but none of the laws below need it.
- `comp k l` is the diagrammatic composite `compose(k, l)` (first k , then l), i.e. the matrix product `kernel_matrix(l) * kernel_matrix(k)`.
- `tensor k l` is `otimes(k, l)`, the outer product of the tables.
- Deterministic kernels are of `Fun f` (Julia `deterministic(X, Y, f)`); the identity, copy, discard, swap and point masses are all of this form, which is what makes the structural laws one-line proofs.

```
namespace FiniteKernelsProofs.Finite

open Finset

/-- A finite kernel 'X → Y' is a real-valued table 'k x y = P(y | x)' ('FiniteKernel.table' in
the outputs-first layout, read as a function of the input first). -/
abbrev Kernel (X Y : Type) : Type := X → Y → ℝ

/-- A state 'I → X' ('state(X, probs)'): a kernel out of the monoidal unit 'Unit'. -/
abbrev State (X : Type) : Type := Kernel Unit X

namespace Kernel

variable {X Y Z W X₁ X₂ X₃ Y₁ Y₂ Y₃ Z₁ Z₂ : Type}
```

Predicates

```
/-- 'is_normalized(k)': each column sums to one. -/
def Normalised [Fintype Y] (k : Kernel X Y) : Prop := ∀ x, ∑ y, k x y = 1

/-- All entries are nonnegative. -/
def Nonneg (k : Kernel X Y) : Prop := ∀ x y, 0 ≤ k x y

/-- A stochastic kernel: nonnegative and normalised (a morphism of **FinStoch**). -/
def Stochastic [Fintype Y] (k : Kernel X Y) : Prop := Nonneg k ∧ Normalised k

/-- A deterministic kernel in the sense of the Julia tests: every column is a point mass. -/
def IsDeterministic [DecidableEq Y] (k : Kernel X Y) : Prop :=
  ∀ x, ∃ y, ∀ y', k x y' = if y' = y then 1 else 0

/-- A state is a point mass ('point_mass(X, a)') at some 'a'. -/
def IsPointMass [DecidableEq X] (p : State X) : Prop :=
  ∃ a, ∀ x, p () x = if x = a then 1 else 0
```

Constructions

```
/-- Sequential composition 'compose(k, l)' (diagrammatic order): '(k • l)(z | x) = ∑ y k(y | x) l(z |
y)',
the matrix product 'kernel_matrix(l) * kernel_matrix(k)'. -/
def comp [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) : Kernel X Z :=
  fun x z => ∑ y, k x y * l y z
```

```

/-- Parallel composition 'otimes(k, l)': the outer product ' $(k \otimes l)(y_1, y_2 \mid x_1, x_2) = k(y_1 \mid x_1) \otimes l(y_2 \mid x_2)$ '. -/
def tensor (k : Kernel X1 Y1) (l : Kernel X2 Y2) : Kernel (X1 × X2) (Y1 × Y2) :=
  fun x y => k x.1 y.1 * l x.2 y.2

/-- The deterministic kernel 'deterministic(X, Y, f)' induced by a function: ' $P(y \mid x) = [y = f x]$ '. -/
def ofFun [DecidableEq Y] (f : X → Y) : Kernel X Y :=
  fun x y => if y = f x then 1 else 0

/-- The identity kernel 'id(X)' ('identity_kernel'). -/
def idK (X : Type) [DecidableEq X] : Kernel X X := ofFun id

/-- The copy map 'mcopy(X) = Δ(X) : X → X ⊗ X', ' $\Delta(x_1, x_2 \mid x) = [x_1 = x][x_2 = x]$ ' ('copy_kernel'). -/
def copy (X : Type) [DecidableEq X] : Kernel X (X × X) := ofFun fun x => (x, x)

/-- The discard map 'delete(X) = ϕ(X) : X → I', the table of ones ('discard_kernel'). -/
def discard (X : Type) : Kernel X Unit := fun _ => 1

/-- The symmetry 'braid(X, Y) = σ(X, Y) : X ⊗ Y → Y ⊗ X' ('swap_kernel'). -/
def swap (X Y : Type) [DecidableEq X] [DecidableEq Y] : Kernel (X × Y) (Y × X) :=
  ofFun (Equiv.prodComm X Y)

/-- The point mass 'point_mass(X, a) = dirac(X, a) : I → X'. -/
def pointMass [DecidableEq X] (a : X) : State X := ofFun fun _ => a

/-- Push a state through a kernel ('apply(k, p) = compose(p, k)'). -/
def push [Fintype X] (k : Kernel X Y) (p : State X) : State Y := comp p k

```

Structural reindexing kernels (the monoidal coherence data).

FiniteKernels.jl is strict (Julia tuples are flat), so these are all identities there; Mathlib's MonoidalCategory is not strict, and its fields refer to them by name.

```

/-- The associator 'α_X Y Z : (X ⊗ Y) ⊗ Z → X ⊗ (Y ⊗ Z)' as a kernel. -/
def assoc (X Y Z : Type) [DecidableEq X] [DecidableEq Y] [DecidableEq Z] :
  Kernel ((X × Y) × Z) (X × (Y × Z)) := ofFun (Equiv.prodAssoc X Y Z)

/-- The left unitor 'λ_X : I ⊗ X → X' as a kernel. -/
def leftUnitor (X : Type) [DecidableEq X] : Kernel (Unit × X) X := ofFun (Equiv.punitProd X)

/-- The inverse left unitor '(λ_X).inv : X → I ⊗ X' as a kernel. -/
def leftUnitorInv (X : Type) [DecidableEq X] : Kernel X (Unit × X) := ofFun (Equiv.punitProd X).symm

/-- The right unitor 'ρ_X : X ⊗ I → X' as a kernel. -/
def rightUnitor (X : Type) [DecidableEq X] : Kernel (X × Unit) X := ofFun (Equiv.prodPUnit X)

/-- The inverse right unitor '(ρ_X).inv : X → X ⊗ I' as a kernel. -/
def rightUnitorInv (X : Type) [DecidableEq X] : Kernel X (X × Unit) := ofFun (Equiv.prodPUnit X).symm

/-- Mathlib's 'tensorμ X X Y Y : (X ⊗ X) ⊗ (Y ⊗ Y) → (X ⊗ Y) ⊗ (X ⊗ Y)', the middle swap
'id(X) ⊗ σ(X, Y) ⊗ id(Y)' of Catlab's coherence axiom for 'mcopy'. -/
def tensorμ (X Y : Type) [DecidableEq X] [DecidableEq Y] :
  Kernel ((X × X) × (Y × Y)) ((X × Y) × (X × Y)) :=
  ofFun fun p => ((p.1.1, p.2.1), (p.1.2, p.2.2))

/-- The unique map 'I ⊗ I → I' ('(λ_I).hom'). -/
def unitMul : Kernel (Unit × Unit) Unit := ofFun fun _ => ()

```

Pointwise evaluation lemmas

```

theorem ext_iff {k l : Kernel X Y} : k = l  $\leftrightarrow$   $\forall x y, k x y = l x y$  := by
  simp [funext_iff]

@[simp] theorem comp_apply [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) (x : X) (z : Z) :
  comp k l x z =  $\sum y, k x y * l y z$  := rfl

@[simp] theorem tensor_apply (k : Kernel X1 Y1) (l : Kernel X2 Y2) (x : X1  $\times$  X2) (y : Y1  $\times$  Y2) :
  tensor k l x y = k x.1 y.1 * l x.2 y.2 := rfl

@[simp] theorem ofFun_apply [DecidableEq Y] (f : X  $\rightarrow$  Y) (x : X) (y : Y) :
  ofFun f x y = if y = f x then 1 else 0 := rfl

@[simp] theorem discard_apply (x : X) (u : Unit) : discard X x u = 1 := rfl

theorem idK_apply [DecidableEq X] (x y : X) : idK X x y = if y = x then 1 else 0 := rfl

theorem copy_apply [DecidableEq X] (x x1 x2 : X) :
  copy X x (x1, x2) = if x1 = x  $\wedge$  x2 = x then 1 else 0 := by
  simp [copy, Prod.ext_iff]

theorem swap_apply [DecidableEq X] [DecidableEq Y] (x : X) (y : Y) (y' : Y) (x' : X) :
  swap X Y (x, y) (y', x') = if y' = y  $\wedge$  x' = x then 1 else 0 := by
  simp [swap, Prod.ext_iff]

theorem pointMass_apply [DecidableEq X] (a x : X) : pointMass a () x = if x = a then 1 else 0 := rfl

/-- 'discard' is the deterministic kernel of the unique map to 'Unit'. -/
theorem discard_eq_ofFun : discard X = ofFun fun _  $\Rightarrow$  () := by
  funext x u; simp

```

The deterministic-kernel calculus

Every structural morphism is ofFun $_$, and ofFun is a functor from functions to kernels that is compatible with tensor. Together with the two lemmas for composing an arbitrary kernel with ofFun on either side, these make the monoidal and comonoid laws mechanical.

```

/-- Precomposing with a deterministic kernel reindexes the input. -/
theorem comp_ofFun_left [Fintype Y] [DecidableEq Y] (f : X  $\rightarrow$  Y) (l : Kernel Y Z) :
  comp (ofFun f) l = fun x z  $\Rightarrow$  l (f x) z := by
  funext x z
  simp [comp, ite_mul, Finset.sum_ite_eq']

/-- Postcomposing with the deterministic kernel of an equivalence reindexes the output. -/
theorem comp_ofEquiv_right [Fintype Y] [DecidableEq Z] (k : Kernel X Y) (e : Y  $\simeq$  Z) :
  comp k (ofFun e) = fun x z  $\Rightarrow$  k x (e.symm z) := by
  funext x z
  simp only [comp_apply, ofFun_apply, mul_ite, mul_one, mul_zero]
  rw [Finset.sum_eq_single (e.symm z)]
  · simp
  · intro b _ hb
    rw [if_neg]
    exact fun h  $\Rightarrow$  hb (e.symm_apply_eq.mpr h).symm
  · simp

/-- Postcomposing with a general deterministic kernel pushes mass forward. -/
theorem comp_ofFun_right [Fintype Y] [DecidableEq Z] (k : Kernel X Y) (g : Y  $\rightarrow$  Z) :
  comp k (ofFun g) = fun x z  $\Rightarrow$   $\sum y, \text{if } z = g y \text{ then } k x y \text{ else } 0$  := by
  funext x z
  simp [comp]

theorem ofFun_comp_ofFun [Fintype Y] [DecidableEq Y] [DecidableEq Z] (f : X  $\rightarrow$  Y) (g : Y  $\rightarrow$  Z) :
  comp (ofFun f) (ofFun g) = ofFun (g  $\circ$  f) := by
  rw [comp_ofFun_left]; rfl

theorem tensor_ofFun [DecidableEq Y1] [DecidableEq Y2] (f : X1  $\rightarrow$  Y1) (g : X2  $\rightarrow$  Y2) :

```

```

    tensor (ofFun f) (ofFun g) = ofFun (Prod.map f g) := by
funext x y
rcases x with ⟨x₁, x₂⟩; rcases y with ⟨y₁, y₂⟩
simp only [tensor_apply, ofFun_apply, Prod.map_apply, Prod.mk.injEq]
split_ifs <|> simp_all

theorem ofFun_congr [DecidableEq Y] {f g : X → Y} (h : ∀ x, f x = g x) : ofFun f = ofFun g := by
rw [funext h]

```

Normalisation and nonnegativity are preserved

```

theorem Normalised.comp [Fintype Y] [Fintype Z] {k : Kernel X Y} {l : Kernel Y Z}
(hk : Normalised k) (hl : Normalised l) : Normalised (comp k l) := by
intro x
have hl' : ∀ y, ∑ z, l y z = 1 := hl
simp only [comp_apply]
rw [Finset.sum_comm]
simp [← Finset.mul_sum, hl']
exact hk x

theorem Normalised.tensor [Fintype Y₁] [Fintype Y₂] {k : Kernel X₁ Y₁} {l : Kernel X₂ Y₂}
(hk : Normalised k) (hl : Normalised l) : Normalised (tensor k l) := by
intro x
simp only [tensor_apply]
rw [Fintype.sum_prod_type, ← Finset.sum_mul_sum, hk, hl, one_mul]

theorem Normalised.ofFun [Fintype Y] [DecidableEq Y] (f : X → Y) : Normalised (ofFun f) := by
intro x
simp [Finset.sum_ite_eq']

theorem Normalised.idK [Fintype X] [DecidableEq X] : Normalised (idK X) := Normalised.ofFun _
theorem Normalised.copy [Fintype X] [DecidableEq X] : Normalised (copy X) := Normalised.ofFun _
theorem Normalised.discard : Normalised (discard X) := by intro x; simp
theorem Normalised.swap [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
Normalised (swap X Y) := Normalised.ofFun _
theorem Normalised.pointMass [Fintype X] [DecidableEq X] (a : X) : Normalised (pointMass a) :=
Normalised.ofFun _

theorem Nonneg.comp [Fintype Y] {k : Kernel X Y} {l : Kernel Y Z} (hk : Nonneg k) (hl : Nonneg l) :
Nonneg (comp k l) := fun x z ⇒
Finset.sum_nonneg fun y _ ⇒ mul_nonneg (hk x y) (hl y z)

theorem Nonneg.tensor {k : Kernel X₁ Y₁} {l : Kernel X₂ Y₂} (hk : Nonneg k) (hl : Nonneg l) :
Nonneg (tensor k l) := fun _ _ ⇒ mul_nonneg (hk _ _) (hl _ _)

theorem Nonneg.ofFun [DecidableEq Y] (f : X → Y) : Nonneg (ofFun f) := by
intro _ _; simp only [ofFun_apply]; split_ifs <|> norm_num

theorem Nonneg.discard : Nonneg (discard X) := fun _ _ ⇒ zero_le_one

theorem Stochastic.comp [Fintype Y] [Fintype Z] {k : Kernel X Y} {l : Kernel Y Z}
(hk : Stochastic k) (hl : Stochastic l) : Stochastic (comp k l) :=
⟨hk.1.comp hl.1, hk.2.comp hl.2⟩

theorem Stochastic.tensor [Fintype Y₁] [Fintype Y₂] {k : Kernel X₁ Y₁} {l : Kernel X₂ Y₂}
(hk : Stochastic k) (hl : Stochastic l) : Stochastic (tensor k l) :=
⟨hk.1.tensor hl.1, hk.2.tensor hl.2⟩

theorem Stochastic.ofFun [Fintype Y] [DecidableEq Y] (f : X → Y) : Stochastic (ofFun f) :=
⟨Nonneg.ofFun f, Normalised.ofFun f⟩

```

Category laws (Julia testset “category laws”)

```

theorem comp_assoc [Fintype Y] [Fintype Z] (k : Kernel X Y) (l : Kernel Y Z) (m : Kernel Z W) :
  comp (comp k l) m = comp k (comp l m) := by
  funext x w
  simp only [comp_apply, Finset.sum_mul, Finset.mul_sum]
  rw [Finset.sum_comm]
  simp [mul_assoc]

theorem idK_comp [Fintype X] [DecidableEq X] (k : Kernel X Y) : comp (idK X) k = k := by
  rw [idK, comp_ofFun_left]; rfl

theorem comp_idK [Fintype Y] [DecidableEq Y] (k : Kernel X Y) : comp k (idK Y) = k := by
  rw [idK]; exact comp_ofEquiv_right k (Equiv.refl Y)

```

Monoidal laws (Julia testset “monoidal laws”)

```

/-- Interchange: 'compose(otimes(k, l), otimes(k', l')) = otimes(compose(k, k'), compose(l, l'))'. -/
theorem comp_tensor [Fintype Y1] [Fintype Y2] (k : Kernel X1 Y1) (l : Kernel X2 Y2)
  (k' : Kernel Y1 Z1) (l' : Kernel Y2 Z2) :
  comp (tensor k l) (tensor k' l') = tensor (comp k k') (comp l l') := by
  funext x z
  simp only [comp_apply, tensor_apply, Fintype.sum_prod_type, Finset.sum_mul_sum]
  exact Finset.sum_congr rfl fun y1 _ => Finset.sum_congr rfl fun y2 _ => by ring

/-- 'otimes(id(X), id(Y)) = id(otimes(X, Y))'. -/
theorem tensor_idK [DecidableEq X] [DecidableEq Y] : tensor (idK X) (idK Y) = idK (X × Y) := by
  simp only [idK, tensor_ofFun]; rfl

/-- Associativity of the tensor, up to the associator:
'otimes(otimes(k, l), m) · α = α · otimes(k, otimes(l, m))'. -/
theorem tensor_assoc [Fintype X1] [Fintype X2] [Fintype X3] [Fintype Y1] [Fintype Y2] [Fintype Y3]
  [DecidableEq X1] [DecidableEq X2] [DecidableEq X3] [DecidableEq Y1] [DecidableEq Y2]
  [DecidableEq Y3] (k : Kernel X1 Y1) (l : Kernel X2 Y2) (m : Kernel X3 Y3) :
  comp (tensor (tensor k l) m) (assoc Y1 Y2 Y3) = comp (assoc X1 X2 X3) (tensor k (tensor l m)) :=
  by
  rw [assoc, assoc, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  rcases x with ⟨⟨x1, x2⟩, x3⟩; rcases y with ⟨y1, y2, y3⟩
  simp [mul_assoc]

/-- Right unit of the tensor, up to the right unitor: 'otimes(k, id(I)) · ρ = ρ · k'. -/
theorem tensor_unit_right [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (k : Kernel X Y) :
  comp (tensor k (idK Unit)) (rightUnitor Y) = comp (rightUnitor X) k := by
  rw [rightUnitor, rightUnitor, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [idK]

/-- Left unit of the tensor, up to the left unitor: 'otimes(id(I), k) · λ = λ · k'. -/
theorem tensor_unit_left [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (k : Kernel X Y) :
  comp (tensor (idK Unit) k) (leftUnitor Y) = comp (leftUnitor X) k := by
  rw [leftUnitor, leftUnitor, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [idK]

/-- The symmetry is an involution: 'braid(X, Y) · braid(Y, X) = id(X ⊗ Y)'. -/
theorem swap_swap [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  comp (swap X Y) (swap Y X) = idK (X × Y) := by
  rw [swap, swap, ofFun_comp_ofFun, idK]; rfl

/-- Naturality of the symmetry: 'otimes(k, l) · braid = braid · otimes(l, k)'. -/
theorem tensor_swap [Fintype X1] [Fintype X2] [Fintype Y1] [Fintype Y2] [DecidableEq X1]
  [DecidableEq X2] [DecidableEq Y1] [DecidableEq Y2] (k : Kernel X1 Y1) (l : Kernel X2 Y2) :
  comp (tensor k l) (swap Y1 Y2) = comp (swap X1 X2) (tensor l k) := by
  rw [swap, swap, comp_ofEquiv_right, comp_ofFun_left]

```

```

funext x y
simp [mul_comm]

/-- The hexagon identity ('SymmetricCategory'/'BraidedCategory.hexagon_forward'), which is the
associator-explicit form of the Julia test
'braid(X ⊗ Y, Z) = (id(X) ⊗ braid(Y, Z)) • (braid(X, Z) ⊗ id(Y))'. -/
theorem hexagon [Fintype X] [Fintype Y] [Fintype Z] [DecidableEq X] [DecidableEq Y]
[DecidableEq Z] :
  comp (assoc X Y Z) (comp (swap X (Y × Z)) (assoc Y Z X)) =
    comp (tensor (swap X Y) (idK Z)) (comp (assoc Y X Z) (tensor (idK Y) (swap X Z))) := by
  simp only [assoc, swap, idK, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- 'braid(X, I) = id(X)' up to the unitors. -/
theorem swap_unit_right [Fintype X] [DecidableEq X] :
  comp (swap X Unit) (leftUnitor X) = rightUnitor X := by
  simp only [swap, leftUnitor, rightUnitor, ofFun_comp_ofFun]
  rfl

```

Comonoid laws (Julia testsets “comonoid laws” and “coherence of copy and discard”).

Each is named after the Mathlib field it instantiates; see Theory/Correspondence.lean.

```

/-- Coassociativity, 'ComonObj.comul_assoc': 'Δ > (X < Δ) = Δ > (Δ > X) > α'. -/
theorem copy_assoc [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (idK X) (copy X)) =
    comp (comp (copy X) (tensor (copy X) (idK X))) (assoc X X X) := by
  simp only [copy, idK, assoc, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- Left counit, 'ComonObj.counit_comul': 'Δ > (ε > X) = (λ_ X).inv'. -/
theorem copy_discard_left [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (discard X) (idK X)) = leftUnitorInv X := by
  simp only [copy, idK, discard_eq_ofFun, leftUnitorInv, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- Right counit, 'ComonObj.comul_counit': 'Δ > (X < ε) = (ρ_ X).inv'. -/
theorem copy_discard_right [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (idK X) (discard X)) = rightUnitorInv X := by
  simp only [copy, idK, discard_eq_ofFun, rightUnitorInv, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- The strict form of the counit laws checked in Julia: 'Δ(X) • (ϕ(X) ⊗ id(X)) = id(X)'. -/
theorem copy_discard_left_strict [Fintype X] [DecidableEq X] :
  comp (comp (copy X) (tensor (discard X) (idK X))) (leftUnitor X) = idK X := by
  rw [copy_discard_left, leftUnitorInv, leftUnitor, ofFun_comp_ofFun, idK]
  exact ofFun_congr fun x => by simp

/-- 'Δ(X) • (id(X) ⊗ ϕ(X)) = id(X)'. -/
theorem copy_discard_right_strict [Fintype X] [DecidableEq X] :
  comp (comp (copy X) (tensor (idK X) (discard X))) (rightUnitor X) = idK X := by
  rw [copy_discard_right, rightUnitorInv, rightUnitor, ofFun_comp_ofFun, idK]
  exact ofFun_congr fun x => by simp

/-- Cocommutativity, 'IsCommComonObj.comul_comm': 'Δ > β = Δ'. -/
theorem copy_swap [Fintype X] [DecidableEq X] : comp (copy X) (swap X X) = copy X := by
  simp only [copy, swap, ofFun_comp_ofFun]
  rfl

/-- Coherence of copy with the tensor, 'CopyDiscardCategory.copy_tensor':
'Δ[X × Y] = (Δ[X] ⊗ Δ[Y]) > tensorμ', Catlab's 'Δ(A ⊗ B) = (Δ(A) ⊗ Δ(B)) • (id(A) ⊗ σ(A,B) ⊗ id(B))'.
-/-
theorem copy_prod [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  copy (X × Y) = comp (tensor (copy X) (copy Y)) (tensorμ X Y) := by
  simp only [copy, tensorμ, tensor_ofFun, ofFun_comp_ofFun]

```

```

rfl

/-- Coherence of discard with the tensor, 'CopyDiscardCategory.discard_tensor':
' $\varepsilon[X \otimes Y] = (\varepsilon[X] \otimes \varepsilon[Y]) \gg (\lambda\_ I).hom$ ', Catlab's ' $\diamond(A \otimes B) = \diamond(A) \otimes \diamond(B)$ '. -/
theorem discard_prod [Fintype X] [Fintype Y] :
  discard (X × Y) = comp (tensor (discard X) (discard Y)) unitMul := by
  funext x u
  simp [unitMul]

/-- 'CopyDiscardCategory.copy_unit': ' $\Delta[I] = (\lambda\_ I).inv$ ', Catlab's ' $\Delta(munit()) = id(munit())$ '. -/
theorem copy_unit : copy Unit = leftUnitorInv Unit := rfl

/-- 'CopyDiscardCategory.discard_unit': ' $\varepsilon[I] = 1\ I$ ', Catlab's ' $\diamond(munit()) = id(munit())$ '. -/
theorem discard_unit : discard Unit = idK Unit := by
  funext x u; simp [idK]

/-- Every deterministic kernel is a comonoid homomorphism: it commutes with copy
('Deterministic.copy_natural', the equation the Julia test checks for 'deterministic(X, Y, f)'). -/
theorem ofFun_copy_natural [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (f : X → Y) :
  comp (ofFun f) (copy Y) = comp (copy X) (tensor (ofFun f) (ofFun f)) := by
  simp only [copy, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- ... and with discard ('Deterministic.discard_natural'). -/
theorem ofFun_discard_natural [Fintype X] [Fintype Y] [DecidableEq Y] (f : X → Y) :
  comp (ofFun f) (discard Y) = discard X := by
  rw [comp_ofFun_left]; rfl

end Kernel

end FiniteKernelsProofs.Finite

```

FiniteKernelsProofs.Finite.Laws

```

import FiniteKernelsProofs.Finite.Kernel
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Linarith

```

The two facts that `FiniteKernels.jl` pins numerically in `test/test_laws.jl` and that separate a Markov category from a cartesian one, proved exactly for the finite model of `Finite/Kernel`.lean:

- **Discard naturality iff normalised** (`comp_discard_eq_discard_iff`): the Markov axiom $f \cdot \diamond(B) == \diamond(A)$ holds for a table precisely when it is normalised. Julia testset “discard naturality iff normalised”, including the $\begin{bmatrix} 0.5 & 0.5 \\ 1.0 & 0.5 \end{bmatrix}$ counterexample (`comp_discard_apply` gives the column sums $\begin{bmatrix} 1.5 \\ 1.0 \end{bmatrix}$).
- **Copy naturality iff deterministic** (`comp_copy_eq_iff_isDeterministic`): for a normalised kernel, $f \cdot \Delta(B) == \Delta(A) \cdot (f \otimes f)$ holds iff every column is a point mass. Julia: “copy is natural for deterministic kernels only”.
- **Copy versus tensor** (`comp_copy_eq_tensor_iff_isPointMass`): the special case of a state $p : I \rightarrow X$; `compose(p, mcopy(X)) = otimes(p, p)` (modulo $I \otimes I \cong I$, which is `copy Unit`) iff p is a point mass. Julia testset “copy once is not two samples (SPEC section 6)”.

The one piece of real arithmetic that exists is `exists_pointMass_of_mul`: if $q \times x * q \times x' = [x = x'] \cdot q \times x$ and $\sum q = 1$, then q is an indicator. From $q \times x * q \times x = q \times x$ one gets $q \times x \in \{0, 1\}$; the sum forces at least one 1; and $q \times x * q \times a = 0$ for $x \neq a$ forces the rest to be 0.

```

namespace FiniteKernelsProofs.Finite.Kernel

open Finset

variable {X Y : Type}

```

Discard naturality iff normalised

```

/-- 'compose(k, delete(Y))' is the table of column sums. -/
theorem comp_discard_apply [Fintype Y] (k : Kernel X Y) (x : X) (u : Unit) :
  comp k (discard Y) x u =  $\sum y, k \ x \ y$  := by
  simp

/-- The Markov axiom ' $f \cdot \phi(B) == \phi(A)$ ' holds for a table iff the table is normalised. -/
theorem comp_discard_eq_discard_iff [Fintype Y] (k : Kernel X Y) :
  comp k (discard Y) = discard X  $\leftrightarrow$  Normalised k := by
  constructor
  · intro h x
    have := congrFun (congrFun h x) ()
    simp using this
  · intro h
    funext x u
    simp [h x]

/-- Discard is natural for normalised kernels ('MarkovCategory.discard_natural'). -/
theorem Normalised.comp_discard [Fintype Y] {k : Kernel X Y} (hk : Normalised k) :
  Kernel.comp k (Kernel.discard Y) = Kernel.discard X :=
  (comp_discard_eq_discard_iff k).mpr hk

/-- A normalised state has total mass one ('state_discard' in 'BayesianNetworksProofs'). -/
theorem Normalised.state_discard [Fintype X] {p : State X} (hp : Normalised p) :
  Kernel.comp p (Kernel.discard X) = Kernel.idK Unit := by
  rw [hp.comp_discard, discard_unit]

```

The indicator lemma

```

/-- If ' $q \ x \ * \ q \ x' = [x = x'] \ q \ x'$ ' and ' $\sum q = 1$ ' then ' $q$ ' is the indicator of a single point. -/
theorem exists_pointMass_of_mul [Fintype X] [DecidableEq X] (q : X  $\rightarrow$   $\mathbb{R}$ ) (hsum :  $\sum x, q \ x = 1$ )
  (h :  $\forall x \ x', q \ x \ * \ q \ x' = \text{if } x = x' \text{ then } q \ x \text{ else } 0$ ) :
   $\exists a, \forall x, q \ x = \text{if } x = a \text{ then } 1 \text{ else } 0$  := by
  have h01 :  $\forall x, q \ x = 0 \vee q \ x = 1$  := by
    intro x
    have hx := h x x
    rw [if_pos rfl] at hx
    have :  $q \ x \ * \ (q \ x - 1) = 0$  := by rw [mul_sub, mul_one, hx, sub_self]
    rcases mul_eq_zero.mp this with h0 | h1
    · exact Or.inl h0
    · exact Or.inr (sub_eq_zero.mp h1)
  have hex :  $\exists a, q \ a = 1$  := by
    by_contra hne
    push Not at hne
    have hzero :  $\forall x, q \ x = 0$  := fun x  $\Rightarrow$  (h01 x).resolve_right (hne x)
    have :  $\sum x, q \ x = 0$  := Finset.sum_eq_zero fun x _  $\Rightarrow$  hzero x
    rw [this] at hsum
    exact zero_ne_one hsum
  obtain ⟨a, ha⟩ := hex
  refine ⟨a, fun x  $\Rightarrow$  ?_⟩
  by_cases hxa : x = a
  · subst hxa; simp [ha]
  · have hx := h x a
    rw [if_neg hxa, ha, mul_one] at hx
    simp [hxa, hx]

/-- Conversely, an indicator satisfies the product identity. -/
theorem mul_eq_of_pointMass [DecidableEq X] (a x x' : X) :
  (if x = a then (1 :  $\mathbb{R}$ ) else 0) * (if x' = a then 1 else 0) =
  if x = x' then (if x = a then 1 else 0) else 0 := by
  by_cases hx : x = a
  · subst hx
    by_cases hx' : x' = x
    · subst hx'; simp

```

- `simp [hx', Ne.symm hx']`
- `by_cases hx' : x' = a`
- `subst hx'; simp [hx]`
- `simp [hx, hx']`

Copy naturality iff deterministic

```

/-- 'compose(k, mcopy(Y))' puts the column of 'k' on the diagonal (draw once, then copy). -/
theorem comp_copy_apply [Fintype Y] [DecidableEq Y] (k : Kernel X Y) (x : X) (y₁ y₂ : Y) :
  comp k (copy Y) x (y₁, y₂) = if y₁ = y₂ then k x y₁ else 0 := by
  simp only [comp_apply, copy, ofFun_apply, Prod.mk.injEq, mul_ite, mul_one, mul_zero]
  by_cases h : y₁ = y₂
  • subst h
    simp
  • rw [if_neg h]
    apply Finset.sum_eq_zero
    intro y _
    rw [if_neg]
    rintro ⟨rfl, rfl⟩
    exact h rfl

/-- 'compose(mcopy(X), otimes(k, k))' is the outer product of the column with itself
(draw twice, independently). -/
theorem copy_comp_tensor_apply [Fintype X] [DecidableEq X] (k : Kernel X Y) (x : X) (y₁ y₂ : Y) :
  comp (copy X) (tensor k k) x (y₁, y₂) = k x y₁ * k x y₂ := by
  simp [copy]

/-- For a normalised kernel, 'f • Δ(B) == Δ(A) • (f ⊗ f)' holds iff 'f' is deterministic. -/
theorem comp_copy_eq_iff_isDeterministic [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
  (k : Kernel X Y) (hk : Normalised k) :
  comp k (copy Y) = comp (copy X) (tensor k k) ↔ IsDeterministic k := by
  constructor
  • intro h x
    refine exists_pointMass_of_mul (k x) (hk x) fun y₁ y₂ => ?_
    have := congrFun (congrFun h x) (y₁, y₂)
    rw [comp_copy_apply, copy_comp_tensor_apply] at this
    exact this.symm
  • intro h
    funext x y
    rcases y with ⟨y₁, y₂⟩
    obtain ⟨a, ha⟩ := h x
    rw [comp_copy_apply, copy_comp_tensor_apply, ha y₁, ha y₂]
    exact (mul_ite_eq_of_pointMass a y₁ y₂).symm

/-- Deterministic kernels are exactly the 'ofFun f' (Julia 'deterministic(X, Y, f)'). -/
theorem isDeterministic_iff_exists_ofFun [DecidableEq Y] (k : Kernel X Y) :
  IsDeterministic k ↔ ∃ f : X → Y, k = ofFun f := by
  constructor
  • intro h
    choose f hf using h
    exact ⟨f, funext fun x => funext fun y => hf x y⟩
  • rintro ⟨f, rfl⟩ x
    exact ⟨f x, fun _ => rfl⟩

/-- Deterministic kernels are normalised. -/
theorem IsDeterministic.normalised [Fintype Y] [DecidableEq Y] {k : Kernel X Y}
  (h : IsDeterministic k) : Normalised k := by
  obtain ⟨f, rfl⟩ := (isDeterministic_iff_exists_ofFun k).mp h
  exact Normalised.ofFun f

```

Copy versus tensor for states

```

theorem isPointMass_iff_isDeterministic [DecidableEq X] (p : State X) :

```

```

    IsPointMass p ↔ IsDeterministic p := by
  constructor
  · rintro ⟨a, ha⟩ _; exact ⟨a, ha⟩
  · intro h; exact h ()

theorem isPointMass_iff_exists_pointMass [DecidableEq X] (p : State X) :
  IsPointMass p ↔ ∃ a, p = pointMass a := by
  constructor
  · rintro ⟨a, ha⟩
    exact ⟨a, funext fun u ⇒ funext fun x ⇒ by cases u; exact ha x⟩
  · rintro ⟨a, rfl⟩
    exact ⟨a, fun x ⇒ rfl⟩

/-- **Copy once is not two samples.** For a normalised state 'p', drawing once and copying
('compose(p, mcopy(X))') equals drawing twice independently ('otimes(p, p)', with the domain
'I ⊗ I' identified with 'I' by 'copy Unit') iff 'p' is a point mass. -/
theorem comp_copy_eq_tensor_iff_isPointMass [Fintype X] [DecidableEq X] (p : State X)
  (hp : Normalised p) :
  comp p (copy X) = comp (copy Unit) (tensor p p) ↔ IsPointMass p := by
  rw [comp_copy_eq_iff_isDeterministic p hp, isPointMass_iff_isDeterministic]

/-- The same statement entrywise, without the 'I ⊗ I ≅ I' reindexing:
'copied.table == independent.table' iff 'p' is a point mass. -/
theorem comp_copy_eq_tensor_iff_isPointMass' [Fintype X] [DecidableEq X] (p : State X)
  (hp : Normalised p) :
  (∀ x x', comp p (copy X) () (x, x') = tensor p p ((), ()) (x, x')) ↔ IsPointMass p := by
  rw [← comp_copy_eq_tensor_iff_isPointMass p hp]
  constructor
  · intro h
    funext u y
    rcases y with ⟨x, x'⟩
    cases u
    rw [copy_comp_tensor_apply]
    exact h x x'
  · intro h x x'
    have := congrFun (congrFun h ()) (x, x')
    rw [copy_comp_tensor_apply] at this
    exact this

/-- Point masses do satisfy the equation (Julia: 'compose(q, mcopy(X)) ≈ otimes(q, q)' for
'q = point_mass(X, l)'). -/
theorem pointMass_comp_copy [Fintype X] [DecidableEq X] (a : X) :
  comp (pointMass a) (copy X) = comp (copy Unit) (tensor (pointMass a) (pointMass a)) :=
  (comp_copy_eq_tensor_iff_isPointMass _ (Normalised.pointMass a)).mpr ⟨a, fun _ ⇒ rfl⟩

/-- The uniform state on two points is not a point mass, so for it the equation fails
(the Julia 'state(X, [0.2, 0.3, 0.5])' example, in its smallest form). -/
example : ¬ IsPointMass (fun _ _ ⇒ (1 / 2 : ℝ) : State (Fin 2)) := by
  rintro ⟨a, ha⟩
  have := ha a
  norm_num at this

example :
  comp (fun _ _ ⇒ (1 / 2 : ℝ) : State (Fin 2)) (copy (Fin 2)) ≠
  comp (copy Unit) (tensor (fun _ _ ⇒ (1 / 2 : ℝ)) (fun _ _ ⇒ (1 / 2 : ℝ))) := by
  rw [Ne, comp_copy_eq_tensor_iff_isPointMass _ (fun _ _ ⇒ by simp)]
  rintro ⟨a, ha⟩
  have := ha a
  norm_num at this

end FiniteKernelsProofs.Finite.Kernel

```

FiniteKernelsProofs.Theory.Correspondence

```
import Mathlib.CategoryTheory.CopyDiscardCategory.Basic
import Mathlib.CategoryTheory.CopyDiscardCategory.Deterministic
import Mathlib.CategoryTheory.MarkovCategory.Basic
```

Dictionary between the GATlab theories used by MarkovCategories.jl (src/theory.jl) and Mathlib's CopyDiscardCategory / MarkovCategory, with the theorem of the finite model (Finite/Kernel.lean, Finite/Laws.lean) that instantiates each axiom.

This module spans two Julia packages: the finite model it instantiates the axioms in belongs to FiniteKernels.jl, which holds these proofs, while the GATlab side of the dictionary now lives in MarkovCategories.jl (its src/theory.jl and src/finstoch_model.jl); FiniteKernels.jl itself carries no Catlab or GATlab dependency and names the same operations compose_kernel, tensor_kernel, identity_kernel, copy_kernel, discard_kernel and swap_kernel.

ThCopyDiscardCategory is MarkovCategories.jl's alias for Catlab's ThMonoidalCategoryWithDiagonals (Catlab/src/theories/Monoidal.jl, "Cartesian category" section); ThMarkovCategory adds one axiom, the naturality of delete.

Comonoid structure

- $\text{Copy mcopy}(A) :: A \rightarrow A \otimes A$ is Mathlib's $\text{ComonObj.comul}(\Delta[X])$ and the finite kernel copy X ; delete is $\text{ComonObj.counit}(\epsilon[X])$ and discard X .
- Coassociativity $\Delta(A) \cdot (\Delta(A) \otimes \text{id}(A)) == \Delta(A) \cdot (\text{id}(A) \otimes \Delta(A))$ is $\text{ComonObj.comul_assoc}$, proved by Kernel.copy_assoc .
- The two counit laws are $\text{ComonObj.counit_comul}$ and $\text{ComonObj.comul_counit}$, proved by copy_discard_left and $\text{copy_discard_right}$ (with $_strict$ variants).
- Cocommutativity $\Delta(A) \cdot \sigma(A, A) == \Delta(A)$ is $\text{IsCommComonObj.comul_comm}$, proved by copy_swap .

Tensor coherence

- $\Delta(A \otimes B) == (\Delta(A) \otimes \Delta(B)) \cdot (\text{id}(A) \otimes \sigma(A, B) \otimes \text{id}(B))$ is $\text{CopyDiscardCategory.copy_tensor}$, proved by copy_prod .
- $\diamond(A \otimes B) == \diamond(A) \otimes \diamond(B)$ is $\text{CopyDiscardCategory.discard_tensor}$, proved by discard_prod .
- $\Delta(\text{munit}()) == \text{id}(\text{munit}())$ and $\diamond(\text{munit}()) == \text{id}(\text{munit}())$ are $\text{CopyDiscardCategory.copy_unit}$ and $\text{CopyDiscardCategory.discard_unit}$, proved by copy_unit and discard_unit .
- The inherited symmetric monoidal structure corresponds to SymmetricCategory and the laws comp_tensor , tensor_assoc , tensor_unit_* , swap_swap , tensor_swap , hexagon .

The Markov axiom and the non-axiom

- $f \cdot \diamond(B) == \diamond(A) \dashv [f :: (A \rightarrow B)]$ is $\text{MarkovCategory.discard_natural}$. In the finite model $\text{comp_discard_eq_discard_iff}$ says it holds exactly when Normalised.
- Copy naturality $f \cdot \Delta(B) == \Delta(A) \cdot (f \otimes f)$ is **not an axiom**: it is the property $\text{Deterministic } f$ ($\text{IsComonHom } f$), characterised by $\text{comp_copy_eq_iff_isDeterministic}$.

Catlab's tensor is strict, so the Mathlib associator, unitors and tensor μ are identities on the Julia side; in the finite model they are the reindexing kernels assoc , leftUnitor , ..., $\text{tensor}\mu$. The generic consequences of these axioms (discard_natural as a theorem, $\text{deterministic_comp}$, $\text{deterministic_copy}$, state_discard) are stated once for the abstract Mathlib classes in the sibling BayesianNetworks.jl proof project's Markov/Basic.lean and are not repeated here. The examples below only check that each Mathlib field has the type the dictionary claims.

```
namespace FiniteKernelsProofs.Theory

open CategoryTheory MonoidalCategory ComonObj CopyDiscardCategory

universe v u

variable {C : Type u} [Category.{v} C] [MonoidalCategory.{v} C]

section CopyDiscard
```

```

variable [CopyDiscardCategory C]

-- Commutative comonoid axioms of 'ThMonoidalCategoryWithDiagonals'.
example (X : C) :  $\Delta[X] \gg (X \triangleleft \Delta[X]) = \Delta[X] \gg (\Delta[X] \triangleright X) \gg (\alpha_- X X X).hom := comul\_assoc X$ 
example (X : C) :  $\Delta[X] \gg (\varepsilon[X] \triangleright X) = (\lambda_- X).inv := counit\_comul X$ 
example (X : C) :  $\Delta[X] \gg (X \triangleleft \varepsilon[X]) = (\rho_- X).inv := comul\_counit X$ 
example (X : C) :  $\Delta[X] \gg (\beta_- X X).hom = \Delta[X] := IsCommComonObj.comul\_comm X$ 

-- Coherence axioms.
-- (' $\Delta[X \otimes Y]$ ' must be read with the 'CopyDiscardCategory.comonObj' instance, not the generic
-- tensor product of comonoids 'Comon.instComonObjTensorObj'; the axiom says the two agree.)
example (X Y : C) :
  let I : ComonObj (X  $\otimes$  Y) := CopyDiscardCategory.comonObj (X  $\otimes$  Y)
   $\Delta[X \otimes Y] = (\Delta[X] \otimes_m \Delta[Y]) \gg tensor_\mu X X Y Y := copy\_tensor X Y$ 
example (X Y : C) :
  let I : ComonObj (X  $\otimes$  Y) := CopyDiscardCategory.comonObj (X  $\otimes$  Y)
   $\varepsilon[X \otimes Y] = (\varepsilon[X] \otimes_m \varepsilon[Y]) \gg (\lambda_- (1_- C)).hom := discard\_tensor X Y$ 
example :  $\Delta[1_- C] = (\lambda_- (1_- C)).inv := copy\_unit$ 
example :  $\varepsilon[1_- C] = 1 (1_- C) := discard\_unit$ 

-- Copy naturality is not an axiom; it is the definition of 'Deterministic'.
example {X Y : C} (f : X  $\rightarrow$  Y) [Deterministic f] : f  $\gg \Delta[Y] = \Delta[X] \gg (f \otimes_m f) :=
  Deterministic.copy\_natural f$ 

end CopyDiscard

section Markov

variable [MarkovCategory C]

-- The one axiom 'ThMarkovCategory' adds.
example {X Y : C} (f : X  $\rightarrow$  Y) : f  $\gg \varepsilon[Y] = \varepsilon[X] := MarkovCategory.discard\_natural f$ 

end Markov

end FiniteKernelsProofs.Theory

```

FiniteKernelsProofs.Theory.FinStoch

```

import FiniteKernelsProofs.Finite.Laws
import Mathlib.CategoryTheory.MarkovCategory.Basic

```

FinStoch as a Mathlib Markov category (SPEC §6, §10.3–§10.5).

Objects are finite types, morphisms are nonnegative normalised real kernels, and the tensor is the Cartesian product of state types with the independent product of kernels. Empty objects are permitted: there is no stochastic map from an inhabited type to an empty type, but this is not an obstruction to the category or its Markov structure.

The associators, unitors and symmetry are deterministic reindexings. Their coherence proofs reduce to equality of the underlying functions, not to additional axioms. In particular copy is **not** assumed natural for arbitrary kernels.

This packages the equations in `Finite/Kernel.lean`; it does not prove a correspondence with Julia floating-point arrays, and it does not make open Bayesian-network syntax a category.

```

set_option autoImplicit false

namespace FiniteKernelsProofs

open CategoryTheory Finite

/-- Finite state types with decidable equality, including the empty type. -/
structure FinStoch where
  carrier : Type
  [fintype : Fintype carrier]

```

```

[decEq : DecidableEq carrier]

attribute [instance] FinStoch.fintype FinStoch.decEq

namespace FinStoch

instance instCategory : Category FinStoch where
  Hom X Y := {k : Kernel X.carrier Y.carrier // Kernel.Stochastic k}
  id X := ⟨Kernel.idK X.carrier, Kernel.Stochastic.ofFun id⟩
  comp k l := ⟨Kernel.comp k.1 l.1, k.2.comp l.2⟩
  id_comp k := Subtype.ext (Kernel.idK_comp k.1)
  comp_id k := Subtype.ext (Kernel.comp_idK k.1)
  assoc k l m := Subtype.ext (Kernel.comp_assoc k.1 l.1 m.1)

variable {X Y Z : FinStoch}

/-- A function induces a stochastic, deterministic morphism. -/
def det (f : X.carrier → Y.carrier) : X → Y :=
  ⟨Kernel.ofFun f, Kernel.Stochastic.ofFun f⟩

/-- A state-space equivalence induces an isomorphism of stochastic kernels. -/
def isoOfEquiv (e : X.carrier ≅ Y.carrier) : X ≅ Y where
  hom := det e
  inv := det e.symm
  hom_inv_id := Subtype.ext (by
    change Kernel.comp (Kernel.ofFun e) (Kernel.ofFun e.symm) = Kernel.idK X.carrier
    rw [Kernel.ofFun_comp_ofFun]
    exact Kernel.ofFun_congr fun x ⇒ e.symm_apply_apply x)
  inv_hom_id := Subtype.ext (by
    change Kernel.comp (Kernel.ofFun e.symm) (Kernel.ofFun e) = Kernel.idK Y.carrier
    rw [Kernel.ofFun_comp_ofFun]
    exact Kernel.ofFun_congr fun y ⇒ e.apply_symm_apply y)

instance instMonoidalCategoryStruct : MonoidalCategoryStruct FinStoch where
  tensorObj X Y := ⟨X.carrier × Y.carrier⟩
  tensorUnit := ⟨Unit⟩
  tensorHom f g := ⟨Kernel.tensor f.1 g.1, f.2.tensor g.2⟩
  whiskerLeft X _ _ g :=
    ⟨Kernel.tensor (Kernel.idK X.carrier) g.1, (Kernel.Stochastic.ofFun id).tensor g.2⟩
  whiskerRight f Y :=
    ⟨Kernel.tensor f.1 (Kernel.idK Y.carrier), f.2.tensor (Kernel.Stochastic.ofFun id)⟩
  associator X Y Z := isoOfEquiv (Equiv.prodAssoc X.carrier Y.carrier Z.carrier)
  leftUnitor X := isoOfEquiv (Equiv.punitProd X.carrier)
  rightUnitor X := isoOfEquiv (Equiv.prodPUnit X.carrier)

open MonoidalCategory

instance instMonoidalCategory : MonoidalCategory FinStoch :=
  MonoidalCategory.ofTensorHom
    (id_tensorHom_id := fun _ _ ⇒ Subtype.ext Kernel.tensor_idK)
    (id_tensorHom := by intros; rfl)
    (tensorHom_id := by intros; rfl)
    (tensorHom_comp_tensorHom := fun f g f' g' ⇒
      Subtype.ext (Kernel.comp_tensor f.1 g.1 f'.1 g'.1))
    (associator_naturality := fun f g h ⇒ Subtype.ext (Kernel.tensor_assoc f.1 g.1 h.1))
    (leftUnitor_naturality := fun f ⇒ Subtype.ext (Kernel.tensor_unit_left f.1))
    (rightUnitor_naturality := fun f ⇒ Subtype.ext (Kernel.tensor_unit_right f.1))
    (pentagon := by
      intro W X Y Z
      apply Subtype.ext
      change Kernel.comp (Kernel.tensor (Kernel.ofFun _) (Kernel.idK _))
        (Kernel.comp (Kernel.ofFun _) (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _))) =
        Kernel.comp (Kernel.ofFun _) (Kernel.ofFun _)
      simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
      rfl)
    (triangle := by

```

```

    intro X Y
    apply Subtype.ext
    change Kernel.comp (Kernel.ofFun _)
      (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _)) =
        Kernel.tensor (Kernel.ofFun _) (Kernel.idK _)
    simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl)

instance instSymmetricCategory : SymmetricCategory FinStoch where
braiding X Y := isoOfEquiv (Equiv.prodComm X.carrier Y.carrier)
braiding_naturality_left := by
  intro X Y f Z
  apply Subtype.ext
  exact Kernel.tensor_swap f.1 (Kernel.idK _)
braiding_naturality_right := by
  intro X Y Z f
  apply Subtype.ext
  exact Kernel.tensor_swap (Kernel.idK _) f.1
hexagon_forward := by
  intro X Y Z
  apply Subtype.ext
  exact Kernel.hexagon
hexagon_reverse := by
  intro X Y Z
  apply Subtype.ext
  change Kernel.comp (Kernel.ofFun _)
    (Kernel.comp (Kernel.ofFun _) (Kernel.ofFun _)) =
      Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _))
        (Kernel.comp (Kernel.ofFun _) (Kernel.tensor (Kernel.ofFun _) (Kernel.idK _)))
  simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
  rfl
symmetry := by
  intro X Y
  apply Subtype.ext
  exact Kernel.swap_swap

instance instComonObj (X : FinStoch) : ComonObj X where
count := ⟨Kernel.discard X.carrier, Kernel.Nonneg.discard, Kernel.Normalised.discard⟩
comul := det fun x => (x, x)
count_comul := Subtype.ext Kernel.copy_discard_left
comul_count := Subtype.ext Kernel.copy_discard_right
comul_assoc := Subtype.ext (by
  change Kernel.comp (Kernel.copy X.carrier)
    (Kernel.tensor (Kernel.idK _) (Kernel.copy _)) =
      Kernel.comp (Kernel.copy X.carrier)
        (Kernel.comp (Kernel.tensor (Kernel.copy _) (Kernel.idK _)) (Kernel.assoc _ _ _))
  simpa only [Kernel.comp_assoc] using (Kernel.copy_assoc (X := X.carrier)))

instance instIsCommComonObj (X : FinStoch) : IsCommComonObj X where
comul_comm := Subtype.ext Kernel.copy_swap

open scoped ComonObj

/-- The structural 'tensorμ' of Mathlib is the middle-swap reindexing kernel. -/
theorem tensorμ_val (X Y : FinStoch) :
  (MonoidalCategory.tensorμ X X Y Y).val = Kernel.tensorμ X.carrier Y.carrier := by
  simp only [MonoidalCategory.tensorμ]
  change Kernel.comp (Kernel.ofFun _)
    (Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _))
      (Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.tensor (Kernel.ofFun _) (Kernel.idK _)))
        (Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _)) (Kernel.ofFun _)))) = _
  simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun, Kernel.tensorμ]
  rfl

instance instMarkovCategory : MarkovCategory FinStoch where
copy_tensor X Y := Subtype.ext (by

```

```

change Kernel.copy (X.carrier × Y.carrier) =
  Kernel.comp (Kernel.tensor (Kernel.copy X.carrier) (Kernel.copy Y.carrier))
    (MonoidalCategory.tensor μ X X Y Y).val
rw [tensor μ_val]
exact Kernel.copy_prod)
discard_tensor X Y := Subtype.ext Kernel.discard_prod
copy_unit := Subtype.ext Kernel.copy_unit
discard_unit := Subtype.ext Kernel.discard_unit
discard_natural f := Subtype.ext (Kernel.Normalised.comp_discard f.2.2)

/-- The abstract Markov discard law uses exactly the normalisation carried by each hom. -/
theorem discard_natural (f : X → Y) : f > ε[Y] = ε[X] :=
  MarkovCategory.discard_natural f

/-- Copy naturality still characterises deterministic kernels; it is not a Markov axiom. -/
theorem copy_natural_iff (f : X → Y) :
  f > Δ[Y] = Δ[X] > (f ⊗m f) ↔ Kernel.IsDeterministic f.val := by
  rw [Subtype.ext_iff]
  exact Kernel.comp_copy_eq_iff_isDeterministic f.1 f.2.2

/-- Empty objects are allowed, but normalisation rules out a map into one from a nonempty
object. No unmentioned inhabitation assumption is used by the category instances. -/
theorem no_hom_to_empty [Nonempty X.carrier] [IsEmpty Y.carrier] (f : X → Y) : False := by
  obtain ⟨x⟩ := ⟨Nonempty X.carrier⟩
  have h := f.2.2 x
  simp at h

end FinStoch

end FiniteKernelsProofs

```

Column-major multi-axis arrays

```

import Mathlib.Logic.Equiv.Fin.Basic
import Mathlib.Algebra.BigOperators.Fin
import Mathlib.Data.List.Rotate
import Mathlib.Tactic.Ring

```

Julia stores an $\text{Array}\{T, N\}$ as one flat vector in **column-major** order: the first axis varies fastest. This module models that storage exactly, with no floating point and values in an arbitrary type.

- A shape is a list of axis sizes $ns = [n_1, \dots, n_N]$ ($\text{size}(\text{table})$); $\text{MIdx } ns$ is the type of in-bounds multi-indices, one $\text{Fin } n_k$ per axis (0-based), and $ns.\text{prod}$ is $\text{length}(\text{table})$.
- $\text{lin idx} = i_1 + n_1 * (i_2 + n_2 * (i_3 + \dots))$ is the 0-based linear position. lin_eq_sum_stride gives the stride form $\sum_k i_k * \prod_{\{j < k\}} n_j$, and $\text{juliaLinearIndex_coords}$ shows that the 1-based fold $_linear_index$ of `FiniteKernels.jl` (`src/kernels.jl`) computes $\text{lin idx} + 1$.
- $\text{toFlat } ns : \text{MIdx } ns \simeq \text{Fin } ns.\text{prod}$ has value $\text{lin}(\text{toFlat_val})$: **the linear index is a bijection between multi-indices and flat positions**. $\text{Tensor } \alpha \text{ } ns := \text{Fin } ns.\text{prod} \rightarrow \alpha$ is a flat table, and Tensor.equivFun identifies flat tables with functions of the multi-index.
- MIdx.append concatenates multi-indices $((y \dots, x \dots))$, and $\text{lin}(\text{append } y \ x) = \text{lin } y + ns.\text{prod} * \text{lin } x$ (lin_append): the flat table of shape $ns ++ ms$ is a column-major $ns.\text{prod} \times ms.\text{prod}$ matrix (reshape).
- MIdx.next is the column-major odometer: $\text{lin}(\text{next idx}) = (\text{lin idx} + 1) \% ns.\text{prod}$.

The labels of `FiniteAxis` are not modelled: an index is a label position (`label_index` minus one). The statements are about index arithmetic, not about Julia execution.

```

namespace FiniteKernelsProofs.Layout

/-- In-bounds multi-indices of a shape, one 0-based coordinate per axis, first axis first. -/
def MIdx : List N → Type
| [] ⇒ Unit
| n :: ns ⇒ Fin n × MIdx ns

```

```

namespace MIdx

/-- The empty multi-index of the zero-dimensional shape. -/
def nil : MIdx [] := ()

/-- Prepend a coordinate on a new first axis. -/
def cons {n : ℕ} {ns : List ℕ} (i : Fin n) (r : MIdx ns) : MIdx (n :: ns) := (i, r)

/-- The coordinate of the first axis. -/
def head {n : ℕ} {ns : List ℕ} (idx : MIdx (n :: ns)) : Fin n := Prod.fst idx

/-- The coordinates of the remaining axes. -/
def tail {n : ℕ} {ns : List ℕ} (idx : MIdx (n :: ns)) : MIdx ns := Prod.snd idx

@[simp] theorem head_cons {n : ℕ} {ns : List ℕ} (i : Fin n) (r : MIdx ns) :
  head (cons i r) = i := rfl

@[simp] theorem tail_cons {n : ℕ} {ns : List ℕ} (i : Fin n) (r : MIdx ns) :
  tail (cons i r) = r := rfl

@[simp] theorem cons_head_tail {n : ℕ} {ns : List ℕ} (idx : MIdx (n :: ns)) :
  cons (head idx) (tail idx) = idx := rfl

theorem ext_nil (a b : MIdx []) : a = b := rfl

theorem ext_cons {n : ℕ} {ns : List ℕ} {a b : MIdx (n :: ns)} (h1 : head a = head b)
  (h2 : tail a = tail b) : a = b := Prod.ext h1 h2

instance instDecidableEq : (ns : List ℕ) → DecidableEq (MIdx ns)
| [] => inferInstanceAs (DecidableEq Unit)
| n :: ns => @instDecidableEqProd (Fin n) (MIdx ns) _ (instDecidableEq ns)

/-- The coordinate list '(i₁, ..., i_N)' (0-based), as Julia's 'Tuple(CartesianIndex)' .- 1'. -/
def coords : {ns : List ℕ} → MIdx ns → List ℕ
| [], _ => []
| _ :: _, idx => (head idx : ℕ) :: coords (tail idx)

@[simp] theorem coords_nil (idx : MIdx []) : coords idx = [] := rfl

@[simp] theorem coords_cons {n : ℕ} {ns : List ℕ} (i : Fin n) (r : MIdx ns) :
  coords (cons i r) = (i : ℕ) :: coords r := rfl

theorem length_coords : {ns : List ℕ} → (idx : MIdx ns) → (coords idx).length = ns.length
| [], _ => rfl
| _ :: _, idx => by simp [coords, length_coords (tail idx)]

/-- A multi-index is determined by its coordinate list. -/
theorem coords_injective : {ns : List ℕ} → Function.Injective (@coords ns)
| [], a, b, _ => ext_nil a b
| _ :: _, a, b, h => by
  simp only [coords, List.cons.injEq] at h
  exact ext_cons (Fin.ext h.1) (coords_injective h.2)

/-- Concatenation '(y..., x...)' of multi-indices. -/
def append : {ns ms : List ℕ} → MIdx ns → MIdx ms → MIdx (ns ++ ms)
| [], _, _, x => x
| _ :: _, _, y, x => cons (head y) (append (tail y) x)

/-- The prefix of a multi-index on 'ns ++ ms'. -/
def fst : {ns ms : List ℕ} → MIdx (ns ++ ms) → MIdx ns
| [], _, _ => ()
| _ :: _, _, z => cons (head z) (fst (ns := _) (tail z))

/-- The suffix of a multi-index on 'ns ++ ms'. -/
def snd : {ns ms : List ℕ} → MIdx (ns ++ ms) → MIdx ms

```

```

| [], _, z ⇒ z
| _ :: _, _, z ⇒ snd (ns := _) (tail z)

theorem fst_append : {ns ms : List ℕ} → (y : MIdx ns) → (x : MIdx ms) → fst (append y x) = y
| [], _, _, _ ⇒ rfl
| _ :: _, _, y, x ⇒ by
  show cons (head y) (fst (append (tail y) x)) = y
  rw [fst_append]; rfl

theorem snd_append : {ns ms : List ℕ} → (y : MIdx ns) → (x : MIdx ms) → snd (append y x) = x
| [], _, _, _ ⇒ rfl
| _ :: _, _, y, x ⇒ snd_append (tail y) x

theorem append_fst_snd : {ns ms : List ℕ} → (z : MIdx (ns ++ ms)) → append (fst z) (snd z) = z
| [], _, _ ⇒ rfl
| _ :: ns, ms, z ⇒ by
  show cons (head z) (append (fst (ns := ns) (ms := ms) (tail z)) (snd (tail z))) = z
  rw [append_fst_snd]; rfl

/-- Multi-indices of 'ns ++ ms' are pairs of multi-indices. -/
def appendEquiv (ns ms : List ℕ) : MIdx ns × MIdx ms ≃ MIdx (ns ++ ms) where
  toFun p := append p.1 p.2
  invFun z := (fst z, snd z)
  left_inv p := by simp [fst_append, snd_append]
  right_inv z := append_fst_snd z

theorem coords_append : {ns ms : List ℕ} → (y : MIdx ns) → (x : MIdx ms) →
  coords (append y x) = coords y ++ coords x
| [], _, _, _ ⇒ rfl
| _ :: _, _, y, x ⇒ by
  show (head y : ℕ) :: coords (append (tail y) x) = ((head y : ℕ) :: coords (tail y)) ++ coords x
  rw [coords_append]; rfl

```

The linear index

```

/-- The 0-based column-major position: ' $i_1 + n_1 * (i_2 + n_2 * (...))$ '. -/
def lin : {ns : List ℕ} → MIdx ns → ℕ
| [], _ ⇒ 0
| n :: _, idx ⇒ (head idx : ℕ) + n * lin (tail idx)

@[simp] theorem lin_nil (idx : MIdx []) : lin idx = 0 := rfl

@[simp] theorem lin_cons {n : ℕ} {ns : List ℕ} (i : Fin n) (r : MIdx ns) :
  lin (cons i r) = i + n * lin r := rfl

theorem lin_lt : {ns : List ℕ} → (idx : MIdx ns) → lin idx < ns.prod
| [], _ ⇒ by simp [lin]
| n :: _, idx ⇒ by
  have h1 := lin_lt (tail idx)
  have h2 := (head idx).isLt
  simp only [lin, List.prod_cons]
  calc (head idx : ℕ) + n * lin (tail idx) < n + n * lin (tail idx) := by omega
  _ = n * (lin (tail idx) + 1) := by ring
  _ ≤ n * _ := Nat.mul_le_mul_left _ h1

/-- The column-major stride of axis 'k': the product of the sizes of the axes before it. -/
def stride (ns : List ℕ) (k : ℕ) : ℕ := (ns.take k).prod

/-- **Stride form of the linear index**: ' $\text{lin idx} = \sum_k i_k * \text{stride } k$ ', with
'stride  $k = \prod_{j < k} n_j$ ' (the first axis has stride one). -/
theorem lin_eq_sum_stride : {ns : List ℕ} → (idx : MIdx ns) →
  lin idx =  $\sum k \in \text{Finset.range ns.length}, (\text{coords idx}).\text{getD } k \ 0 * \text{stride } ns \ k$ 
| [], _ ⇒ by simp [lin]
| n :: ns, idx ⇒ by
  rw [show idx = cons (head idx) (tail idx) from rfl, List.length_cons, Finset.sum_range_succ',

```

```

    lin_cons, coords_cons, lin_eq_sum_stride (tail idx), Finset.mul_sum]
simp only [List.getD_cons_succ, List.getD_cons_zero, stride, List.take_succ_cons,
List.prod_cons, List.take_zero, List.prod_nil, mul_one]
rw [add_comm]
congr 1
exact Finset.sum_congr rfl fun k _ => by ring

/-- The 1-based fold of `'_linear_index(dims, idx)' in 'FiniteKernels.jl':
`lin = 1; stride = 1; for (d, i) in zip(dims, idx); lin += (i - 1) * stride; stride *= d; end`. -/
def juliaLinearIndex (dims idx : List ℕ) : ℕ :=
  ((dims.zip idx).foldl (fun acc p => (acc.1 + (p.2 - 1) * acc.2, acc.2 * p.1)) (1, 1)).1

theorem foldl_linear : {ns : List ℕ} → (idx : MIdx ns) → (a s : ℕ) →
  (ns.zip ((coords idx).map (· + 1))).foldl
    (fun acc p => (acc.1 + (p.2 - 1) * acc.2, acc.2 * p.1)) (a, s) =
    (a + s * lin idx, s * ns.prod)
| [], _, a, s => by simp [lin]
| n :: ns, idx, a, s => by
  rw [show idx = cons (head idx) (tail idx) from rfl]
  simp only [coords_cons, List.map_cons, List.zip_cons_cons, List.foldl_cons,
    Nat.add_sub_cancel, lin_cons, List.prod_cons]
  rw [foldl_linear (tail idx)]
  ext <=> simp only <=> ring

/-- **Julia's `'_linear_index' is the column-major position**, shifted to 1-based. -/
theorem juliaLinearIndex_coords {ns : List ℕ} (idx : MIdx ns) :
  juliaLinearIndex ns ((coords idx).map (· + 1)) = lin idx + 1 := by
  simp [juliaLinearIndex, foldl_linear, add_comm]

```

The bijection with flat positions

```

/-- Multi-indices of a shape are in bijection with the flat positions 'Fin ns.prod'. -/
def toFlat : (ns : List ℕ) → MIdx ns ≈ Fin ns.prod
| [] =>
  { toFun := fun _ => ⟨0, by simp⟩
  invFun := fun _ => ()
  left_inv := fun _ => rfl
  right_inv := fun p => Fin.ext (by
    have h : (p : ℕ) < 1 := p.isLt
    show 0 = (p : ℕ)
    omega) }
| n :: ns =>
  ((Equiv.prodCongr (Equiv.refl (Fin n)) (toFlat ns)).trans (Equiv.prodComm _ _)).trans
    (finProdFinEquiv.trans (finCongr (by simp [Nat.mul_comm])))

/-- **The flat position of a multi-index is its column-major linear index.** -/
@[simp] theorem toFlat_val : {ns : List ℕ} → (idx : MIdx ns) → (toFlat ns idx : ℕ) = lin idx
| [], _ => rfl
| n :: ns, idx => by
  have h := toFlat_val (tail idx)
  simp only [toFlat]
  rw [lin]
  exact congrArg _ (congrArg _ h)

theorem lin_injective {ns : List ℕ} : Function.Injective (@lin ns) := by
  intro a b h
  apply (toFlat ns).injective
  exact Fin.ext (by simp using h)

theorem lin_surjective {ns : List ℕ} (p : ℕ) (hp : p < ns.prod) : ∃ idx : MIdx ns, lin idx = p :=
  ⟨(toFlat ns).symm ⟨p, hp⟩, by rw [← toFlat_val, Equiv.apply_symm_apply]⟩

/-- **The column-major linear index is a bijection** from multi-indices onto the flat
positions 'Fin ns.prod' ('Fin (prod(size(table)))'). -/
theorem linearIndex_bijective (ns : List ℕ) :

```

```

Function.Bijective (fun idx : MIdx ns ⇒ (⟨lin idx, lin_lt idx⟩ : Fin ns.prod)) := by
have h : (fun idx : MIdx ns ⇒ (⟨lin idx, lin_lt idx⟩ : Fin ns.prod)) = toFlat ns :=
  funext fun idx ⇒ Fin.ext (toFlat_val idx).symm
rw [h]
exact (toFlat ns).bijective

instance instFintype (ns : List ℕ) : Fintype (MIdx ns) := Fintype.ofEquiv _ (toFlat ns).symm

theorem card (ns : List ℕ) : Fintype.card (MIdx ns) = ns.prod := by
  rw [Fintype.card_congr (toFlat ns), Fintype.card_fin]

/-- **Concatenated multi-indices**: the flat table of shape 'ns ++ ms' is the column-major
'ns.prod × ms.prod' matrix whose entry '[lin y, lin x]' sits at 'lin (y..., x...)'. -/
theorem lin_append : {ns ms : List ℕ} → (y : MIdx ns) → (x : MIdx ms) →
  lin (append y x) = lin y + ns.prod * lin x
| [], _, _ ⇒ by simp [append]
| n :: ns, _, y, x ⇒ by
  show (head y : ℕ) + n * lin (append (tail y) x) = (head y : ℕ) + n * lin (tail y) +
    (n * ns.prod) * lin x
  rw [lin_append]
  ring

```

The column-major odometer

```

/-- The next multi-index in column-major order, wrapping to all zeros after the last. -/
def next : {ns : List ℕ} → MIdx ns → MIdx ns
| [], _ ⇒ ()
| n :: _, idx ⇒
  if h : (head idx : ℕ) + 1 < n then cons ⟨head idx + 1, h⟩ (tail idx)
  else cons ⟨0, lt_of_le_of_lt (Nat.zero_le _) (head idx).isLt⟩ (next (tail idx))

theorem lin_next : {ns : List ℕ} → (idx : MIdx ns) → lin (next idx) = (lin idx + 1) % ns.prod
| [], _ ⇒ by simp [lin]
| n :: ns, idx ⇒ by
  have hi := (head idx).isLt
  have hr := lin_lt (tail idx)
  by_cases h : (head idx : ℕ) + 1 < n
  · have hlt : (head idx : ℕ) + n * lin (tail idx) + 1 < n * ns.prod :=
      calc (head idx : ℕ) + n * lin (tail idx) + 1 < n + n * lin (tail idx) := by omega
        _ = n * (lin (tail idx) + 1) := by ring
        _ ≤ n * ns.prod := Nat.mul_le_mul_left _ hr
    have hn : next idx = cons ⟨head idx + 1, h⟩ (tail idx) := dif_pos h
    rw [hn]
    show ((head idx : ℕ) + 1) + n * lin (tail idx) =
      ((head idx : ℕ) + n * lin (tail idx) + 1) % (n * ns.prod)
    rw [Nat.mod_eq_of_lt hlt]
    ring
  · have he : (head idx : ℕ) + 1 = n := by omega
    have hn : next idx = cons ⟨0, by omega⟩ (next (tail idx)) := dif_neg h
    have hs : (head idx : ℕ) + n * lin (tail idx) + 1 = n * (lin (tail idx) + 1) :=
      calc (head idx : ℕ) + n * lin (tail idx) + 1 = ((head idx : ℕ) + 1) + n * lin (tail idx) :=
        by ring
        _ = n + n * lin (tail idx) := by rw [he]
        _ = n * (lin (tail idx) + 1) := by ring
    rw [hn]
    show 0 + n * lin (next (tail idx)) =
      ((head idx : ℕ) + n * lin (tail idx) + 1) % (n * ns.prod)
    rw [lin_next (tail idx), zero_add, hs, Nat.mul_mod_mul_left]

/-- Stepping the odometer 'p' times from position zero reaches flat position 'p'. -/
theorem lin_iterate_next {ns : List ℕ} (z : MIdx ns) (hz : lin z = 0) (p : ℕ) :
  lin (next^[p] z) = p % ns.prod := by
  induction p with
  | zero ⇒
    rw [Function.iterate_zero, id_eq, hz, Nat.zero_mod]

```

```

| succ p ih =>
  rw [Function.iterate_succ_apply', lin_next, ih, Nat.mod_add_mod]

end MIdx

```

Flat tables

```

/-- A flat table of shape 'ns': 'length(table) = ns.prod' values in column-major order
('vec(table)'). -/
def Tensor (α : Type) (ns : List ℕ) : Type := Fin ns.prod → α

namespace Tensor

variable {α : Type} {ns : List ℕ}

/-- 'table[i₁, ..., i_N]': the value stored at the column-major position of the multi-index. -/
def get (A : Tensor α ns) (idx : MIdx ns) : α := A (MIdx.toFlat ns idx)

/-- The flat table whose entry at every multi-index is 'f idx'. -/
def ofFun (f : MIdx ns → α) : Tensor α ns := fun p => f ((MIdx.toFlat ns).symm p)

@[simp] theorem get_ofFun (f : MIdx ns → α) (idx : MIdx ns) : get (ofFun f) idx = f idx := by
  simp [get, ofFun]

@[simp] theorem ofFun_get (A : Tensor α ns) : ofFun (get A) = A := by
  funext p
  simp [get, ofFun]

theorem ext_get {A B : Tensor α ns} (h : ∀ idx, get A idx = get B idx) : A = B := by
  rw [← ofFun_get A, ← ofFun_get B]
  exact congrArg ofFun (funext h)

/-- Flat column-major tables are exactly functions of the multi-index. -/
def equivFun (α : Type) (ns : List ℕ) : Tensor α ns ≈ (MIdx ns → α) where
  toFun := get
  invFun := ofFun
  left_inv := ofFun_get
  right_inv f := funext (get_ofFun f)

/-- Reading the flat position whose 1-based index is Julia's '_linear_index' fold is 'get'. -/
theorem get_eq_juliaLinearIndex (A : Tensor α ns) (idx : MIdx ns) (p : Fin ns.prod)
  (hp : (p : ℕ) + 1 = MIdx.juliaLinearIndex ns ((MIdx.coords idx).map (· + 1))) :
  get A idx = A p := by
  unfold get
  congr 1
  apply Fin.ext
  rw [MIdx.toFlat_val]
  rw [MIdx.juliaLinearIndex_coords] at hp
  omega

end Tensor

end FiniteKernelsProofs.Layout

```

The outputs-first kernel layout and the (parents..., child) CPT layout

```

import FiniteKernelsProofs.Layout.ColumnMajor
import FiniteKernelsProofs.Finite.Kernel

```

FiniteKernel.table stores a kernel $\text{dom} \rightarrow \text{codom}$ **outputs first**: $\text{size}(\text{table}) = (\text{size}(\text{codom}) \dots, \text{size}(\text{dom}) \dots)$ and $\text{table}[y \dots, x \dots] = P(y \mid x)$ (ADR 0002). User CPTs are (parents..., child), normalised over the last axis, and cpt is the only conversion: $\text{permutedims}(\text{table}, (n + 1, 1, \dots, n))$ one way,

permutedims(table, (2, ..., n + 1, 1)) the other.

- `KernelTable  $\alpha$  cs ds := Tensor  $\alpha$  (cs ++ ds)`; `toKernel T x y = table[y..., x...]` is a function of the input first, the convention of `Finite.Kernel` (for $\alpha = \mathbb{R}$ it is a `Finite.Kernel (MIdx ds) (MIdx cs)`), and `kernelEquiv` shows flat outputs-first tables and kernels are the same data.
- `kernelMatrix_toFlat`: the flat table read as the column-major `length(codom)  $\times$  length(dom)` matrix (`kernel_matrix = reshape(table, ...)`) has entry $P(y \mid x)$ at row `lin y`, column `lin x`.
- `probability_linear`: `probability(k, y, x)` reads `_linear_index(size(table), (iy..., ix...))`, which is `lin (y..., x...) + 1`.
- `cptToKernel / kernelToCpt` model the two `permutedims` calls. They are mutually inverse (`cptEquiv`), so the permutation of flat storage positions `cptPerm` is a bijection, and every entry is preserved: `toKernel_cptToKernel` says that $P(\text{child} = i \mid \text{parents} = x)$ read from the kernel is the CPT entry `table[x..., i]`. `cptToKernel_permutedims` and `kernelToCpt_permutedims` show that they satisfy Julia's `permutedims(A, perm)` contract $B[I] = A[J]$ whenever $J[\text{perm}[k]] = I[k]$, for exactly the tuples $(n + 1, 1, \dots, n)$ and $(2, \dots, n + 1, 1)$ used in `src/kernels.jl`.
- `normalised_cptToKernel_iff`: the kernel is normalised iff every CPT row sums to one over the last (child) axis.

This links layouts and index arithmetic. It does not execute Julia, and the values are exact (any type; $\mathbb{R}$ for normalisation), not IEEE floating point.

```
namespace FiniteKernelsProofs.Layout
```

```
open MIdx
```

```
variable { $\alpha$  : Type}
```

Outputs-first kernel tables

```
/-- 'FiniteKernel.table' for codomain shape 'cs' and domain shape 'ds': outputs first. -/
abbrev KernelTable ( $\alpha$  : Type) (cs ds : List N) : Type := Tensor  $\alpha$  (cs ++ ds)

namespace KernelTable

variable {cs ds : List N}

/-- 'table[y..., x...]' -/
def entry (T : KernelTable  $\alpha$  cs ds) (y : MIdx cs) (x : MIdx ds) :  $\alpha$  := Tensor.get T (append y x)

/-- The kernel stored by an outputs-first table, input first: 'toKernel T x y = P(y | x)' -/
def toKernel (T : KernelTable  $\alpha$  cs ds) : MIdx ds  $\rightarrow$  MIdx cs  $\rightarrow$   $\alpha$  := fun x y  $\Rightarrow$  T.entry y x

/-- The outputs-first table of a kernel. -/
def ofKernel (k : MIdx ds  $\rightarrow$  MIdx cs  $\rightarrow$   $\alpha$ ) : KernelTable  $\alpha$  cs ds :=
  Tensor.ofFun fun z  $\Rightarrow$  k (snd z) (fst z)

@[simp] theorem toKernel_ofKernel (k : MIdx ds  $\rightarrow$  MIdx cs  $\rightarrow$   $\alpha$ ) : toKernel (ofKernel k) = k := by
  funext x y
  simp [toKernel, ofKernel, entry, fst_append, snd_append]

@[simp] theorem ofKernel_toKernel (T : KernelTable  $\alpha$  cs ds) : ofKernel (toKernel T) = T := by
  apply Tensor.ext_get
  intro z
  simp [toKernel, ofKernel, entry, append_fst_snd]

/-- Flat outputs-first tables and kernels (input first) are the same data. -/
def kernelEquiv ( $\alpha$  : Type) (cs ds : List N) :
  KernelTable  $\alpha$  cs ds  $\simeq$  (MIdx ds  $\rightarrow$  MIdx cs  $\rightarrow$   $\alpha$ ) where
  toFun := toKernel
  invFun := ofKernel
  left_inv := ofKernel_toKernel
  right_inv := toKernel_ofKernel

/-- For real entries the stored kernel is a kernel of the finite model 'Finite.Kernel'. -/
```

```

def toFiniteKernel (T : KernelTable R cs ds) : Finite.Kernel (MIdx ds) (MIdx cs) := toKernel T

theorem prod_append (cs ds : List ℕ) : (cs ++ ds).prod = cs.prod * ds.prod := List.prod_append

/-- 'kernel_matrix(k) = reshape(table, length(codom), length(dom))', read column-major. -/
def kernelMatrix (T : KernelTable α cs ds) (r : Fin cs.prod) (c : Fin ds.prod) : α :=
  T ⟨r + cs.prod * c, by
    rw [prod_append]
    have hr := r.isLt
    have hc := c.isLt
    calc (r : ℕ) + cs.prod * c < cs.prod + cs.prod * c := by omega
    _ = cs.prod * (c + 1) := by ring
    _ ≤ cs.prod * ds.prod := Nat.mul_le_mul_left _ hc⟩

/-- '**kernel_matrix' has 'P(y | x)' at row 'lin y', column 'lin x'.** -/
theorem kernelMatrix_toFlat (T : KernelTable α cs ds) (y : MIdx cs) (x : MIdx ds) :
  kernelMatrix T (toFlat cs y) (toFlat ds x) = T.entry y x := by
  unfold kernelMatrix entry Tensor.get
  congr 1
  apply Fin.ext
  simp [lin_append]

/-- '**probability(k, y, x)' reads the entry 'table[y..., x...]'**: Julia's 1-based
'_linear_index(size(table), (Tuple(iy)..., Tuple(ix)...)' is 'lin (y..., x...) + 1'. -/
theorem probability_linear (y : MIdx cs) (x : MIdx ds) :
  juliaLinearIndex (cs ++ ds) ((coords y ++ coords x).map (· + 1)) =
  lin (append y x) + 1 := by
  rw [← coords_append, juliaLinearIndex_coords]

end KernelTable

```

The (parents..., child) CPT layout and cpt

```

/-- A user CPT: shape '(parents..., child)'. -/
abbrev CptTable (α : Type) (ps : List ℕ) (c : ℕ) : Type := Tensor α (ps ++ [c])

/-- The multi-index of a single child axis. -/
def childIdx {c : ℕ} (i : Fin c) : MIdx [c] := cons i nil

@[simp] theorem head_childIdx {c : ℕ} (i : Fin c) : head (childIdx i) = i := rfl

theorem childIdx_head {c : ℕ} (y : MIdx [c]) : childIdx (head y) = y := rfl

namespace CptTable

variable {ps : List ℕ} {c : ℕ}

/-- 'table[x..., i]': the probability of child state 'i' given parent states 'x'. -/
def entry (A : CptTable α ps c) (x : MIdx ps) (i : Fin c) : α :=
  Tensor.get A (append x (childIdx i))

end CptTable

/-- 'cpt(parents, child, table)': 'permutedims(table, (n + 1, 1, ..., n))', child axis first. -/
def cptToKernel {ps : List ℕ} {c : ℕ} (A : CptTable α ps c) : KernelTable α [c] ps :=
  Tensor.ofFun fun z => Tensor.get A (append (snd (ns := [c]) z) (fst (ns := [c]) z))

/-- 'cpt(k)': 'permutedims(table, (2, ..., n + 1, 1))', back to parents first. -/
def kernelToCpt {ps : List ℕ} {c : ℕ} (T : KernelTable α [c] ps) : CptTable α ps c :=
  Tensor.ofFun fun w => Tensor.get T (append (snd (ns := ps) w) (fst (ns := ps) w))

theorem get_cptToKernel {ps : List ℕ} {c : ℕ} (A : CptTable α ps c) (y : MIdx [c])
  (x : MIdx ps) : Tensor.get (cptToKernel A) (append y x) = Tensor.get A (append x y) := by
  simp [cptToKernel, fst_append, snd_append]

```

```

theorem get_kernelToCpt {ps : List ℕ} {c : ℕ} (T : KernelTable α [c] ps) (x : MIdx ps)
  (y : MIdx [c]) : Tensor.get (kernelToCpt T) (append x y) = Tensor.get T (append y x) := by
  simp [kernelToCpt, fst_append, snd_append]

/-- 'cpt' preserves every entry: the kernel built from a CPT gives
'P(child = i | parents = x) = table[x..., i]'. -/
theorem toKernel_cptToKernel {ps : List ℕ} {c : ℕ} (A : CptTable α ps c) (x : MIdx ps)
  (i : Fin c) : KernelTable.toKernel (cptToKernel A) x (childIdx i) = A.entry x i :=
  get_cptToKernel A (childIdx i) x

theorem entry_kernelToCpt {ps : List ℕ} {c : ℕ} (T : KernelTable α [c] ps) (x : MIdx ps)
  (i : Fin c) : (kernelToCpt T).entry x i = KernelTable.toKernel T x (childIdx i) :=
  get_kernelToCpt T x (childIdx i)

@[simp] theorem kernelToCpt_cptToKernel {ps : List ℕ} {c : ℕ} (A : CptTable α ps c) :
  kernelToCpt (cptToKernel A) = A := by
  apply Tensor.ext_get
  intro w
  rw [← append_fst_snd w, get_kernelToCpt, get_cptToKernel]

@[simp] theorem cptToKernel_kernelToCpt {ps : List ℕ} {c : ℕ} (T : KernelTable α [c] ps) :
  cptToKernel (kernelToCpt T) = T := by
  apply Tensor.ext_get
  intro z
  rw [← append_fst_snd (ns := [c]) z, get_cptToKernel, get_kernelToCpt]

/-- The two 'cpt' conversions are mutually inverse. -/
def cptEquiv (α : Type) (ps : List ℕ) (c : ℕ) : CptTable α ps c ≈ KernelTable α [c] ps where
  toFun := cptToKernel
  invFun := kernelToCpt
  left_inv := kernelToCpt_cptToKernel
  right_inv := cptToKernel_kernelToCpt

/-- The permutation of flat storage positions that 'cpt' performs. -/
def cptPerm (ps : List ℕ) (c : ℕ) : Fin (ps ++ [c]).prod ≈ Fin ([c] ++ ps).prod :=
  (toFlat (ps ++ [c])).symm.trans <| (appendEquiv ps [c]).symm.trans <|
  (Equiv.prodComm _ _).trans <| (appendEquiv [c] ps).trans (toFlat ([c] ++ ps))

/-- 'cpt' moves storage by a bijection of positions and copies every value:
the kernel table at position 'q' is the CPT value at position 'cptPerm.symm q'. -/
theorem cptToKernel_apply {ps : List ℕ} {c : ℕ} (A : CptTable α ps c)
  (q : Fin ([c] ++ ps).prod) : cptToKernel A q = A ((cptPerm ps c).symm q) := by
  rfl

```

Julia's permutedims contract

```

/-- 'B = permutedims(A, perm)' (0-based 'perm'): 'size(B, k) = size(A, perm[k])' and
'B[I] = A[J]' whenever 'J[perm[k]] = I[k]' for every axis 'k'. -/
def PermutedimsSpec {ns ms : List ℕ} (perm : ℕ → ℕ) (A : Tensor α ns) (B : Tensor α ms) : Prop :=
  (∀ k < ms.length, ms[k]? = ns[perm k]?) ∧
  ∀ (I : MIdx ms) (J : MIdx ns),
    (∀ k < ms.length, (coords J)[perm k]? = (coords I)[k]?) → Tensor.get B I = Tensor.get A J

/-- The tuple '(n + 1, 1, ..., n)' of 'cpt(parents, child, table)', 0-based. -/
def cptPermTuple (n : ℕ) (k : ℕ) : ℕ := if k = 0 then n else k - 1

/-- The tuple '(2, ..., n + 1, 1)' of 'cpt(k)', 0-based. -/
def uncptPermTuple (n : ℕ) (k : ℕ) : ℕ := if k < n then k + 1 else 0

theorem coords_eq_of_perm {ps : List ℕ} {c : ℕ} (x : MIdx ps) (i : Fin c) (J : MIdx (ps ++ [c]))
  (h : ∀ k < ps.length + 1,
    (coords J)[cptPermTuple ps.length k]? = ((i : ℕ) :: coords x)[k]?) :
  J = append x (childIdx i) := by

```

```

apply coords_injective
rw [coords_append]
apply List.ext_getElem?
intro k
have hJ := length_coords J
have hx := length_coords x
simp only [List.length_append, List.length_cons, List.length_nil] at hJ
rcases lt_trichotomy k ps.length with hk | hk | hk
• have := h (k + 1) (by omega)
  simp only [cptPermTupl, Nat.add_one_ne_zero, if_false, Nat.add_sub_cancel,
    List.getElem?_cons_succ] at this
  have hk' : k < (coords x).length := by omega
  rw [this, List.getElem?_append_left hk']
• subst hk
  have := h 0 (by omega)
  simp only [cptPermTupl, if_true, List.getElem?_cons_zero] at this
  have hle : (coords x).length ≤ ps.length := le_of_eq hx
  rw [this, List.getElem?_append_right hle, hx, Nat.sub_self]
  all_goals rfl
• have h1 : (coords J).length ≤ k := by omega
  have h2 : (coords x ++ coords (childIdx i)).length ≤ k := by
    rw [List.length_append, hx]
    show ps.length + 1 ≤ k
    omega
  rw [List.getElem?_eq_none h1, List.getElem?_eq_none h2]

/-- **'cpt(parents, child, table)' is 'permutedims(table, (n + 1, 1, ..., n))'**. -/
theorem cptToKernel_permutedims {ps : List ℕ} {c : ℕ} (A : CptTable α ps c) :
  PermutedimsSpec (cptPermTupl ps.length) A (cptToKernel A) := by
  constructor
  • intro k hk
    simp only [List.length_append, List.length_cons, List.length_nil] at hk
    rcases Nat.eq_zero_or_pos k with rfl | hk0
    • simp [cptPermTupl]
    • obtain ⟨j, rfl⟩ : ∃ j, k = j + 1 := ⟨k - 1, by omega⟩
      have hj : j < ps.length := by omega
      simp only [cptPermTupl, Nat.add_one_ne_zero, if_false, Nat.add_sub_cancel]
      rw [List.getElem?_append_left (l₂ := [c]) hj]
      simp
  • intro I J hIJ
    have hI : I = append (fst (ns := [c]) I) (snd (ns := [c]) I) := (append_fst_snd I).symm
    have hc : coords I = (head (fst (ns := [c]) I) : ℕ) :: coords (snd (ns := [c]) I) := by
      conv_lhs ⇒ rw [hI]
      rw [coords_append]
      all_goals rfl
    have hJ : J = append (snd (ns := [c]) I) (childIdx (head (fst (ns := [c]) I))) := by
      apply coords_eq_of_perm
      intro k hk
      have hk' : k < ([c] ++ ps).length := by simp; omega
      rw [hIJ k hk', hc]
    rw [hJ]
    conv_lhs ⇒ rw [hI]
    rw [get_cptToKernel, childIdx_head]

/-- **'cpt(k)' is 'permutedims(table, (2, ..., n + 1, 1))'**. -/
theorem kernelToCpt_permutedims {ps : List ℕ} {c : ℕ} (T : KernelTable α [c] ps) :
  PermutedimsSpec (uncptPermTupl ps.length) T (kernelToCpt T) := by
  constructor
  • intro k hk
    simp only [List.length_append, List.length_cons, List.length_nil] at hk
    by_cases hkn : k < ps.length
    • simp only [uncptPermTupl, hkn, if_true]
      rw [List.getElem?_append_left (l₂ := [c]) hkn]
      simp
    • have : k = ps.length := by omega
      subst this

```

```

    simp [uncptPermTuple]
  • intro I J hIJ
  have hI : I = append (fst (ns := ps) I) (snd (ns := ps) I) := (append_fst_snd I).symm
  have hcI : coords I = coords (fst (ns := ps) (ms := [c]) I) ++
    [(head (snd (ns := ps) (ms := [c]) I) : ℕ)] := by
    conv_lhs ⇒ rw [hI]
    rw [coords_append]
    all_goals rfl
  have hy : coords (snd (ns := ps) (ms := [c]) I) = [(head (snd (ns := ps) (ms := [c]) I) : ℕ)] :=
    rfl
  have hx := length_coords (fst (ns := ps) (ms := [c]) I)
  have hJ : J = append (snd (ns := ps) (ms := [c]) I) (fst (ns := ps) (ms := [c]) I) := by
    apply coords_injective
    rw [coords_append]
    apply List.ext_getElem?
    intro k
    have hlen := length_coords J
    simp only [List.length_append, List.length_cons, List.length_nil] at hlen
    rcases Mat.eq_zero_or_pos k with rfl | hk0
    • have := hIJ ps.length (by simp)
      simp only [uncptPermTuple, lt_irrefl, if_false] at this
      have hle : (coords (fst (ns := ps) (ms := [c]) I)).length ≤ ps.length := le_of_eq hx
      rw [this, hcI, List.getElem?_append_right hle, hx, Nat.sub_self, hy]
      all_goals simp
    • obtain ⟨j, rfl⟩ : ∃ j, k = j + 1 := ⟨k - 1, by omega⟩
      by_cases hj : j < ps.length
      • have := hIJ j (by simp; omega)
        simp only [uncptPermTuple, hj, if_true] at this
        have hj' : j < (coords (fst (ns := ps) (ms := [c]) I)).length := by omega
        rw [this, hcI, List.getElem?_append_left hj', hy]
        all_goals simp
      • have h1 : (coords J).length ≤ j + 1 := by omega
        have h2 : (coords (snd (ns := ps) (ms := [c]) I) ++
          coords (fst (ns := ps) (ms := [c]) I)).length ≤ j + 1 := by
          rw [List.length_append, hx]
          show 1 + ps.length ≤ j + 1
          omega
        rw [List.getElem?_eq_none h1, List.getElem?_eq_none h2]
    rw [hJ]
  conv_lhs ⇒ rw [hI]
  rw [get_kernelToCpt]

```

Normalisation

```

/-- **A CPT normalised over its last axis gives a normalised kernel, and conversely.** -/
theorem normalised_cptToKernel_iff {ps: List ℕ} {c : ℕ} (A : CptTable ℝ ps c) :
  Finite.Kernel.Normalised (KernelTable.toFiniteKernel (cptToKernel A)) ↔
  ∀ x, ∑ i : Fin c, A.entry x i = 1 := by
  have hsum : ∀ x, ∑ y : MIdx [c], KernelTable.toFiniteKernel (cptToKernel A) x y =
    ∑ i : Fin c, A.entry x i := by
    intro x
    let e : MIdx [c] ≈ Fin c :=
      { toFun := head, invFun := childIdx, left_inv := childIdx_head, right_inv := fun _ => rfl }
    refine Fintype.sum_equiv e _ (fun i => A.entry x i) fun y => ?_
    exact toKernel_cptToKernel A x (head y)
  simp only [Finite.Kernel.Normalised, hsum]

end FiniteKernelsProofs.Layout

```

The stride-based product of two factors

```
import FiniteKernelsProofs.Layout.ColumnMajor
```

`multiply(f, g)` in `BayesianNetworkInference.jl` (`src/factors.jl`) builds the product over the union of the two scopes without permuting either table:

- `_union_axes` lists the result axes (f's variables, then g's new ones); a factor's axis `j` is the result axis `pa[j]`, so its table has shape `pa.map ns.get`;
- `_result_strides(h, vars)` gives each result axis the column-major stride that h's own table has for that variable, and 0 if h lacks it (`resultStride`, `resultStride_eq`);
- `_product_into!` walks the result's joint states in column-major order with an odometer, updating the two table offsets incrementally (`bump`, `productLoop`), and stores `out[i] = a[ai] * b[bi]`.

Here a factor over result positions `pa` is, semantically, the function $I \mapsto F.get \text{ (project } I \text{ } pa)$ of the joint multi-index (it reads the coordinates of its own axes), and the **named product** is the pointwise product of those functions (`namedProduct`).

- `offset_eq_lin`: for a factor without repeated variables (`pa.Nodup`, which the `Factor` constructor enforces), the result-stride offset $\sum_k I_k * resultStride \ k$ is the position of the projected multi-index in the factor's own table.
- `slotOffset_eq_lin`: the per-slot strides of BayesianNetworks' `_Factor` (one stride per table axis, repeats allowed) compute the same position with no `Nodup` hypothesis, and `resultStride_repeated` shows that the result-stride form is wrong for repeated axes.
- `bump_spec` and `productLoop_eq`: the odometer step moves to `MIdx.next` and keeps each offset equal to its stride sum.
- `productInto_getElem?` / `productInto_eq`: **the list written by the loop is the flat column-major table of the named product**, including the zero-dimensional case (one entry).

Values live in any type with a multiplication (a semiring in practice); this is exact index arithmetic, not a statement about Julia execution or IEEE rounding.

```
namespace FiniteKernelsProofs.Layout
```

```
open MIdx
```

Coordinates and projections

```
/-- The coordinate of axis 'k', as an element of that axis. -/
def MIdx.coord : {ns : List ℕ} → MIdx ns → (k : Fin ns.length) → Fin (ns.get k)
| [], _, k ⇒ k.elim0
| _ :: _, idx, ⟨0, _⟩ ⇒ head idx
| _ :: _, idx, ⟨k + 1, h⟩ ⇒ coord (tail idx) ⟨k, Nat.lt_of_succ_lt_succ h⟩

/-- All coordinates of a multi-index at linear position zero are zero. -/
theorem MIdx.coord_eq_zero_of_lin : {ns : List ℕ} → (z : MIdx ns) → lin z = 0 →
  ∀ k, (coord z k : ℕ) = 0
| [], _, _, k ⇒ k.elim0
| n :: _, z, h, ⟨0, _⟩ ⇒ by
  have h' : (head z : ℕ) + n * lin (tail z) = 0 := h
  show (head z : ℕ) = 0
  omega
| n :: _, z, h, ⟨k + 1, hk⟩ ⇒ by
  have h' : (head z : ℕ) + n * lin (tail z) = 0 := h
  have hn : 0 < n := lt_of_le_of_lt (Nat.zero_le _) (head z).isLt
  have ht : lin (tail z) = 0 := by
    rcases Nat.mul_eq_zero.1 (by omega : n * lin (tail z) = 0) with h'' | h''
    · omega
    · exact h''
  exact coord_eq_zero_of_lin (tail z) ht ⟨k, Nat.lt_of_succ_lt_succ hk⟩

/-- The multi-index of a factor whose axis 'j' is the result axis 'pa[j]': 'I[pa]'. -/
def project {ns : List ℕ} (I : MIdx ns) : (pa : List (Fin ns.length)) → MIdx (pa.map ns.get)
| [] ⇒ nil
```

```

| p :: pa ⇒ cons (coord I p) (project I pa)

variable {α : Type} {ns : List ℕ}

/-- The named (functional) product: at every joint multi-index, the product of the two
factors' entries at their projected multi-indices. -/
def namedProduct [Mul α] (pa pb : List (Fin ns.length)) (F : Tensor α (pa.map ns.get))
  (G : Tensor α (pb.map ns.get)) : Tensor α ns :=
  Tensor.ofFun fun I ⇒ F.get (project I pa) * G.get (project I pb)

```

Strides

```

/-- '_result_strides(h, vars)', recursively: the first slot has stride one and every later slot
the size of the earlier ones times its stride in the tail. -/
def resultStride (ns : List ℕ) : List (Fin ns.length) → Fin ns.length → ℕ
| [], _ ⇒ 0
| p :: pa, i ⇒ if i = p then 1 else ns.get p * resultStride ns pa i

/-- 'resultStride' is literally '_result_strides': the column-major stride of the first
slot of the factor that carries result axis 'i', and '0' when there is none. -/
theorem resultStride_eq (pa : List (Fin ns.length)) (i : Fin ns.length) :
  resultStride ns pa i = if i ∈ pa then ((pa.map ns.get).take (pa.indexOf i)).prod else 0 := by
  induction pa with
  | nil ⇒ simp [resultStride]
  | cons p pa ih ⇒
    by_cases h : i = p
    · subst h
      simp [resultStride]
    · have hne : p ≠ i := Ne.symm h
      simp only [resultStride, h, if_false, ih, List.mem_cons, false_or, List.map_cons,
        List.indexOf_cons_ne _ hne, Nat.succ_eq_add_one, List.take_succ_cons, List.prod_cons]
      split_ifs <|> simp

theorem resultStride_of_notMem (pa : List (Fin ns.length)) (i : Fin ns.length) (h : i ∉ pa) :
  resultStride ns pa i = 0 := by
  rw [resultStride_eq, if_neg h]

/-- The table offset 'Σk Ik * resultStride k' of the result multi-index 'I'. -/
def offset (pa : List (Fin ns.length)) (I : MIdx ns) : ℕ :=
  Σ k : Fin ns.length, (coord I k : ℕ) * resultStride ns pa k

/-- Result strides locate the projected entry when the factor has no repeated variable. -/
theorem offset_eq_lin (pa : List (Fin ns.length)) (hpa : pa.Nodup) (I : MIdx ns) :
  offset pa I = lin (project I pa) := by
  induction pa with
  | nil ⇒ simp [offset, resultStride, project]
  | cons p pa ih ⇒
    obtain ⟨hp, hpa⟩ := List.nodup_cons.1 hpa
    have hterm : ∀ k : Fin ns.length, (coord I k : ℕ) * resultStride ns (p :: pa) k =
      (if k = p then (coord I k : ℕ) else 0) +
      ns.get p * ((coord I k : ℕ) * resultStride ns pa k) := by
      intro k
      by_cases hk : k = p
      · rw [hk]
        simp [resultStride, resultStride_of_notMem pa p hp]
      · simp only [resultStride, hk, if_false, zero_add]
        ring
    unfold offset
    simp only [hterm, Finset.sum_add_distrib, Finset.sum_ite_eq', Finset.mem_univ, if_true,
      ← Finset.mul_sum]
    rw [show (Σ k : Fin ns.length, (coord I k : ℕ) * resultStride ns pa k) = offset pa I from rfl,
      ih hpa]
    rfl

```

```

theorem coords_project (I : MIdx ns) (pa : List (Fin ns.length)) :
  coords (project I pa) = pa.map fun p => (coord I p : ℕ) := by
  induction pa with
  | nil => rfl
  | cons p pa ih =>
    show (coord I p : ℕ) :: coords (project I pa) = _
    rw [ih]
    rfl

/-- **Per-slot strides** ('BayesianNetworks' '_Factor': 'idx += strides[j] * (ci[axes[j]] - 1)')
locate the projected entry for every slot list, repeated result axes included. -/
theorem slotOffset_eq_lin (pa : List (Fin ns.length)) (I : MIdx ns) :
  lin (project I pa) = ∑ j ∈ Finset.range (pa.map ns.get).length,
    (coords (project I pa)).getD j 0 * stride (pa.map ns.get) j :=
  lin_eq_sum_stride _

/-- With a repeated axis the result-stride form reads the wrong entry: for one result axis of
size two and a factor carrying it twice (the diagonal of a '2 × 2' table), the result-stride
offset of state '1' is '1', but the diagonal entry '(1, 1)' sits at position '3'. -/
theorem resultStride_repeated :
  offset (ns := [2]) [(0, by decide), (0, by decide)] (cons (1, by decide) nil) = 1 ∧
  lin (project (ns := [2]) (cons (1, by decide) nil) [(0, by decide), (0, by decide)]) = 3 := by
  decide

```

The odometer loop of _product_into!

```

/-- '∑k posk * sk' over two lists. -/
def dot (pos : List ℕ) (s : List ℤ) : ℤ := (List.zipWith (fun p t => (p : ℤ) * t) pos s).sum

/-- One pass of the inner 'for k in 1:d' loop, for one table offset:
'pos[k] += 1; off += s[k]; pos[k] < sz[k] && break; pos[k] = 0; off -= s[k] * sz[k]'. -/
def bump : List ℕ → List ℕ → List ℤ → ℤ → List ℕ × ℤ
| n :: sz, p :: pos, t :: s, off =>
  if p + 1 < n then ((p + 1) :: pos, off + t)
  else
    let r := bump sz pos s (off + t - t * n)
    (0 :: r.1, r.2)
| _, pos, _, off => (pos, off)

theorem dot_cons (p : ℕ) (pos : List ℕ) (t : ℤ) (s : List ℤ) :
  dot (p :: pos) (t :: s) = p * t + dot pos s := by
  simp [dot]

/-- **The odometer step**: from the coordinates of 'idx' and an offset equal to its stride sum
(plus any constant), 'bump' reaches the coordinates of 'next idx' and its stride sum. -/
theorem bump_spec : {sz : List ℕ} → (idx : MIdx sz) → (s : List ℤ) → s.length = sz.length →
  (off : ℤ) → bump sz (coords idx) s (off + dot (coords idx) s) =
  (coords (next idx), off + dot (coords (next idx)) s)
| [], idx, s, _, off => by
  simp [bump, coords]
| n :: sz, idx, [], hs, _ => by simp at hs
| n :: sz, idx, t :: s, hs, off => by
  have hs' : s.length = sz.length := by simpa using hs
  have hi := (head idx).isLt
  rw [show idx = cons (head idx) (tail idx) from rfl]
  simp only [coords_cons, dot_cons, bump]
  by_cases h : (head idx : ℕ) + 1 < n
  · rw [if_pos h]
    simp only [next, head_cons, tail_cons, h, dite_true, coords_cons, dot_cons,
      Prod.mk.injEq, true_and]
    push_cast
    ring
  · rw [if_neg h]
    have he : (head idx : ℕ) + 1 = n := by omega

```

```

have hoff : off + ((head idx : ℕ) * t + dot (coords (tail idx)) s) + t - t * n =
  off + dot (coords (tail idx)) s := by
  have hn : (n : ℤ) = ((head idx : ℕ) : ℤ) + 1 := by exact_mod_cast he.symm
  rw [hn]
  ring
simp only [hoff, bump_spec (tail idx) s hs' off, next, head_cons, tail_cons, h,
  dite_false, coords_cons, dot_cons, Nat.cast_zero, zero_mul, zero_add]

/-- The loop of `_product_into!`: write `a[ai] * b[bi]`, then advance the shared odometer and
both offsets. -/
def productLoop [Mul α] (sz : List ℕ) (as bs : List ℤ) (a b : ℕ → α) :
  ℕ → List ℕ → ℤ → ℤ → List α
| 0, _, _, _ ⇒ []
| k + 1, pos, ai, bi ⇒
  (a ai.toNat * b bi.toNat) ::
    productLoop sz as bs a b k (bump sz pos as ai).1 (bump sz pos as ai).2 (bump sz pos bs bi).2

theorem productLoop_eq [Mul α] (sz : List ℕ) (as bs : List ℤ) (has : as.length = sz.length)
  (hbs : bs.length = sz.length) (a b : ℕ → α) (k : ℕ) (idx : MIdx sz) :
  productLoop sz as bs a b k (coords idx) (dot (coords idx) as) (dot (coords idx) bs) =
    (List.range k).map fun j ⇒
      a (dot (coords (next^[j] idx)) as).toNat * b (dot (coords (next^[j] idx)) bs).toNat := by
induction k generalizing idx with
| zero ⇒ rfl
| succ k ih ⇒
  have ha := bump_spec idx as has 0
  have hb := bump_spec idx bs hbs 0
  simp only [zero_add] at ha hb
  simp only [productLoop, ha, hb]
  rw [ih, List.range_succ_eq_map, List.map_cons, List.map_map]
  rfl

/-- The coordinates of the all-zero multi-index. -/
theorem coords_of_lin_eq_zero : {sz : List ℕ} → (z : MIdx sz) → lin z = 0 →
  coords z = List.replicate sz.length 0
| [], _, _ ⇒ rfl
| n :: sz, z, h ⇒ by
  have hn : 0 < n := lt_of_le_of_lt (Nat.zero_le _) (head z).isLt
  rw [show z = cons (head z) (tail z) from rfl] at h
  simp only [lin_cons] at h
  have h0 : (head z : ℕ) = 0 := by omega
  have ht : lin (tail z) = 0 := by
    rcases Nat.mul_eq_zero.1 (by omega : n * lin (tail z) = 0) with h' | h'
    · omega
    · exact h'
  rw [coords_cons, h0, coords_of_lin_eq_zero (tail z) ht]
  rfl

/-- `_product_into!(out, sz, a, as, b, bs)` with `pos = zeros`, `ai = bi = 1` (0-based here). -/
def productInto [Mul α] (sz : List ℕ) (as bs : List ℤ) (a b : ℕ → α) : List α :=
  productLoop sz as bs a b sz.prod (List.replicate sz.length 0) 0 0

theorem dot_ofFn {sz : List ℕ} (I : MIdx sz) (f : Fin sz.length → ℕ) :
  dot (coords I) (List.ofFn fun k ⇒ (f k : ℤ)) = ((∑ k, (coord I k : ℕ) * f k : ℕ) : ℤ) := by
induction sz with
| nil ⇒ simp [dot, coords]
| cons n sz ih ⇒
  rw [show I = cons (head I) (tail I) from rfl, List.ofFn_succ, coords_cons, dot_cons]
  show _ = ((∑ k : Fin (sz.length + 1), (coord (cons (head I) (tail I)) k : ℕ) * f k : ℕ) : ℤ)
  rw [Fin.sum_univ_succ, ih (tail I) (fun k ⇒ f k.succ)]
  push_cast
  rfl

/-- The result strides of a factor as the integer vector the loop uses. -/
def strides (ns : List ℕ) (pa : List (Fin ns.length)) : List ℤ :=
  List.ofFn fun k ⇒ (resultStride ns pa k : ℤ)

```

```

theorem dot_strides (pa : List (Fin ns.length)) (hpa : pa.Nodup) (I : MIdx ns) :
  dot (coords I) (strides ns pa) = lin (project I pa) := by
  rw [strides, dot_ofFn, show (Σ k, (coord I k : ℕ) * resultStride ns pa k) = offset pa I
    from rfl, offset_eq_lin pa hpa]

/-- The projection of the all-zero multi-index has linear position zero. -/
theorem lin_project_zero (z : MIdx ns) (hz : lin z = 0) (pa : List (Fin ns.length)) :
  lin (project z pa) = 0 := by
  induction pa with
  | nil => rfl
  | cons p pa ih =>
    show (coord z p : ℕ) + ns.get p * lin (project z pa) = 0
    rw [coord_eq_zero_of_lin z hz p, ih, mul_zero, add_zero]

theorem get_eq_lin (A : Tensor α ns) (idx : MIdx ns) : A.get idx = A ⟨lin idx, lin_lt idx⟩ := by
  unfold Tensor.get
  congr 1
  exact Fin.ext (toFlat_val idx)

theorem length_productLoop [Mul α] (sz : List ℕ) (as bs : List ℤ) (a b : ℕ → α) (k : ℕ)
  (pos : List ℕ) (ai bi : ℤ) : (productLoop sz as bs a b k pos ai bi).length = k := by
  induction k generalizing pos ai bi with
  | zero => rfl
  | succ k ih => simp [productLoop, ih]

/-- **The stride-based product computes the named product, entry by entry.** For factors
without repeated variables whose flat tables 'a', 'b' store 'F', 'G' column-major, the entry
that '_product_into!' writes at the column-major position of the joint multi-index 'I' is
'F[I[pa]] * G[I[pb]]'. -/
theorem productInto_getElem? [Mul α] (pa pb : List (Fin ns.length)) (hpa : pa.Nodup)
  (hpb : pb.Nodup) (F : Tensor α (pa.map ns.get)) (G : Tensor α (pb.map ns.get))
  (a b : ℕ → α) (ha : ∀ J, a (lin J) = F.get J) (hb : ∀ J, b (lin J) = G.get J)
  (I : MIdx ns) :
  (productInto ns (strides ns pa) (strides ns pb) a b)[lin I]? =
  some ((namedProduct pa pb F G).get I) := by
  have hpos : 0 < ns.prod := lt_of_le_of_lt (Nat.zero_le _) (lin_lt I)
  obtain ⟨z, hz⟩ := lin_surjective (ns := ns) 0 hpos
  have hlen : ∀ pa : List (Fin ns.length), (strides ns pa).length = ns.length := by
    intro pa; simp [strides]
  have hstart : productInto ns (strides ns pa) (strides ns pb) a b =
    productLoop ns (strides ns pa) (strides ns pb) a b ns.prod (coords z)
    (dot (coords z) (strides ns pa)) (dot (coords z) (strides ns pb)) := by
    rw [dot_strides pa hpa, dot_strides pb hpb, lin_project_zero z hz, lin_project_zero z hz,
      coords_of_lin_eq_zero z hz]
  all_goals rfl
  rw [hstart, productLoop_eq _ _ _ (hlen pa) (hlen pb), List.getElem?_map,
    List.getElem?_range (lin_lt I), Option.map_some]
  refine congrArg some ?_
  show a (dot (coords (next^[lin I] z)) (strides ns pa)).toNat *
    b (dot (coords (next^[lin I] z)) (strides ns pb)).toNat = _
  have hiter : next^[lin I] z = I := by
    apply lin_injective
    rw [lin_iterate_next z hz, Nat.mod_eq_of_lt (lin_lt I)]
  rw [hiter, dot_strides pa hpa, dot_strides pb hpb, Int.toNat_natCast, Int.toNat_natCast, ha, hb,
    namedProduct, Tensor.get_ofFun]

/-- **The loop output is the flat column-major table of the named product.** -/
theorem productInto_eq [Mul α] (pa pb : List (Fin ns.length)) (hpa : pa.Nodup)
  (hpb : pb.Nodup) (F : Tensor α (pa.map ns.get)) (G : Tensor α (pb.map ns.get))
  (a b : ℕ → α) (ha : ∀ J, a (lin J) = F.get J) (hb : ∀ J, b (lin J) = G.get J) :
  productInto ns (strides ns pa) (strides ns pb) a b =
  List.ofFn (namedProduct pa pb F G) := by
  apply List.ext_getElem?
  intro p
  by_cases hp : p < ns.prod

```

```

• obtain ⟨I, rfl⟩ := lin_surjective (ns := ns) p hp
  rw [productInto_getElem? pa pb hpa hpb F G a b ha hb I, List.getElem?_ofFn,
    dif_pos (lin_lt I), get_eq_lin]
• have h1 : (productInto ns (strides ns pa) (strides ns pb) a b).length ≤ p := by
  rw [productInto, length_productLoop]
  omega
have h2 : (List.ofFn (namedProduct pa pb F G)).length ≤ p := by
  rw [List.length_ofFn]
  omega
rw [List.getElem?_eq_none h1, List.getElem?_eq_none h2]

end FiniteKernelsProofs.Layout

```

Roadmap

```
import FiniteKernelsProofs.Theory.FinStoch
```

The two former holes, `MonoidalCategory FinStoch` and `MarkovCategory FinStoch`, are now proved in `Theory/FinStoch.lean`, imported by the default target and included in the axiom audit. This compatibility module has no unproved declarations.

The layout half of the representation bridge to Julia's arrays is now in the default target: `Layout/ColumnMajor.lean` (column-major storage; the linear index is a bijection onto `Fin (∏ sizes)` and equals `_linear_index`), `Layout/KernelLayout.lean` (outputs-first kernel tables, `kernel_matrix`, `probability`, and the two `cpt` `permutedims` as mutually inverse, entry-preserving conversions) and `Layout/Product.lean` (the result strides and odometer of `BayesianNetworkInference.multiply` compute the named product for factors without repeated variables). Labels (`label_index`) are modelled only as positions.

Remaining: floating-point values and IEEE arithmetic, and any proof that the Julia code executes these definitions (the correspondence is read off the source). `_broadcastable` is not modelled. The `Mathlib` instance by itself does not establish that bridge, and no open-network category or semantic functor is constructed here.

```

namespace FiniteKernelsProofs.Roadmap

/-- Compatibility name for the finite stochastic category now in the default library. -/
abbrev FinStoch := FiniteKernelsProofs.FinStoch

end FiniteKernelsProofs.Roadmap

```